Ch. 0 Introduction. This subject is *app development* not Programming. Learning Outcome:

Mobile App Overview: Understand difference of app dev between mobile and other.

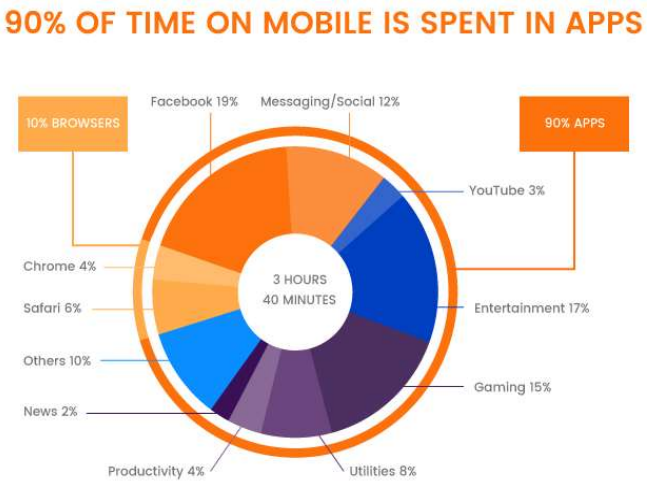
Business Strategy: Identify app requirements and features that boost both user satisfaction & company

Design Pattern: Implement current trends in designing a modern app UI/UX with a reliable architecture

Deliverable Product. Examine technique to release a mobile app to the customer via multiple channels with analytics and error logging.

CHAPTER 1 Introduction to Mobile Development

1.Why Mobile App?



3. Mobile App Platform

Parameters	Native app development	Cross-platform app development	Hybrid app development	Progressive web app development
Definition	These apps run natively on the platform of your choice	Cross-platform app development is about developing one app and publishing it on both Android and iOS together	It is based on the web view which runs on a web application in a native browser	Progressive web apps are hybrids of regular web pages and native apps
Tools and technologies used	Java, Kotlin, Python, etc.	Xamarin, NativeScript, Node.js, etc.	Objective C, Ionic, HTML5, etc.	Javascript, Ruby, CSS, etc.
Advantages	1. Greater user experience 2. Direct access to the device's APIs	1. Single code for multiple platforms 2. Easy to maintain app	1. Only one codebase is required for a hybrid app 2. Access to device's internal APIs and device hardware	1. The same app runs on web and mobile. Hence, it has less expensive upfronts 2. No need to install or update as it is accessible through URL
Performance	Fastest speed and efficiency as it is directly built on OS's platform	The need for an additional abstraction layer and rendering process makes the cross-platform app slower than its native counterpart	Slower than native and cross-platform apps as they are not usually optimized for their platform	Progressive web apps can be decently fast, but their speed is entirely dependent on internet connection

2. Mobile App Characteristics

Battery Life: Limited power capacity

Continuous Changes: OS updates and new hardware are introduced every day

Store Distribution: App should be reviewed and distributed via official stores

Unstable Communication: Not always available with 4G/5G Technology, Wifi, BLE, NFC4

Small Screen Size: 5.9-6.5" screens are the most popular

2.1 Important Characteristics

Short-Duration Activities: The activities tend to be short term, from several seconds to minutes

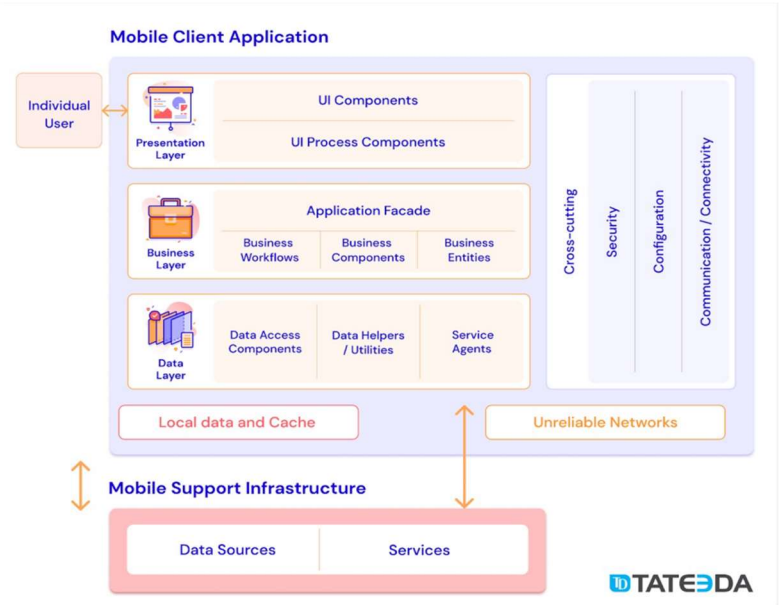
Focused Purpose: Provide services as quickly, efficiently as possible with little or no navigation.

Responsiveness: Must return something immediately upon performing an action on the device.

Customized Data Interaction: Require limiting size of the content being downloaded via the Internet

	React Native	IONIC	Xamarin	Flutter
	A framework for building native apps with React.	One app running on everything.	Complete Binding for the underlying SDKs.	Makes it easy and fast to build beautiful mobile apps.
	77 401 ★	77 401 ★		64 773 ★
Developers	Facebook	Drifty	Microsoft	Google
Language	JavaScript	TypeScript	C#	Dart
Performance	Close to native	Moderate	Moderate	Amazingcontrolle
GUI	Use native UI controllers	HTML, CSS	Use native UI controllers	Use proprietary widgets and deliver UI
Code reusability	90% of code is reusable	98% of code is reusable	98% of code is reusable	50 - 90% of code is reusable
Testing	Mobile device or emulator	Any browser	Mobile device or emulator, test cloud	Mobile device or emulator
Hot reload	Yes	No	No	Yes
Apps	Airbnb, Discord, Instagram	MarketWatch, Pacifica, JustWatch	Olo, Storyo, APX	KlasterMe, PostMuse Reflectly

4. Mobile App Architecture.

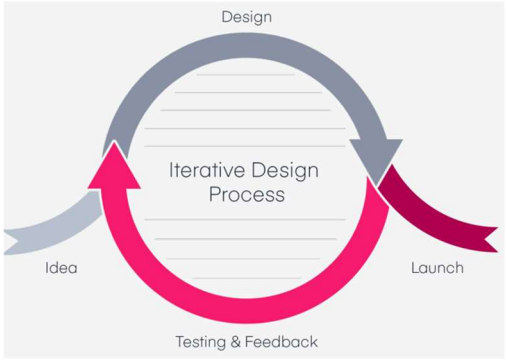


The **most popular multilayer architecture** is the **three-layer architecture**, together with other supported modules

What is a good Mobile App architecture?'

Quality, speed of app performance: well-debugged and smoothly running -> more popular with users
Compatibility with **different types** of devices and mobile **platforms** will enable a wider audience to discover and use your application and this will help you generate more income.
High levels of **scalability** and **adjustability**: is adding new features fast and easy or does it require a massive rethinking of the app structure?
User-friendliness, usability, and efficiency in executing intended user tasks and functions.
Killer features and other features that support competitiveness and **safeguard good market positions** of your app.
Able to **graceful degradation and/or fault-tolerance** for **UX**.
Cost-efficient application **architecture**: is your mobile app designed wisely enough to avoid numerous and costly fixes and revisions in the long run?

5. Mobile App Business



According to Criteo, users view 4.2x more products per session in mobile apps than mobile websites.
Balance your budget and resources: Don't try to satisfy all your customer's demands
Big things often have small beginnings - except for bad things. They start big and then get smaller

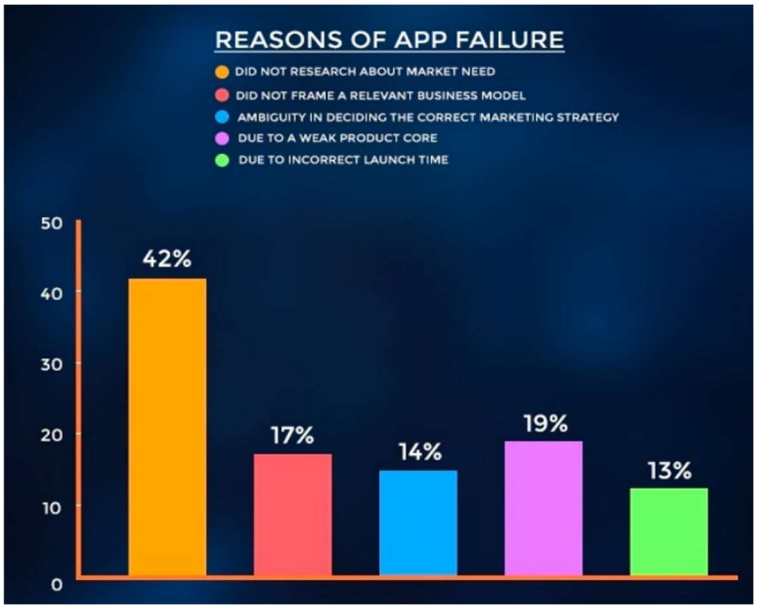
6. Minimum Viable Product (MVP)



MVP is developing a basic version of the app based on **must-have features** and **launching** it in the **market** for **customer validation**

This **helps** businesses achieve **product-market fit** while also helping **clients save money and time.**

Learn from mistakes The mobile app market is saturated with competition. Only 20% of downloaded apps see users return after the first use, whereas 3% of apps remain in use after a month.



Ch2. Mobile Application Design Process

1. Mobile App Quality Evaluation

1.1 Quality Factors: How can we determine a " good" mobile app?

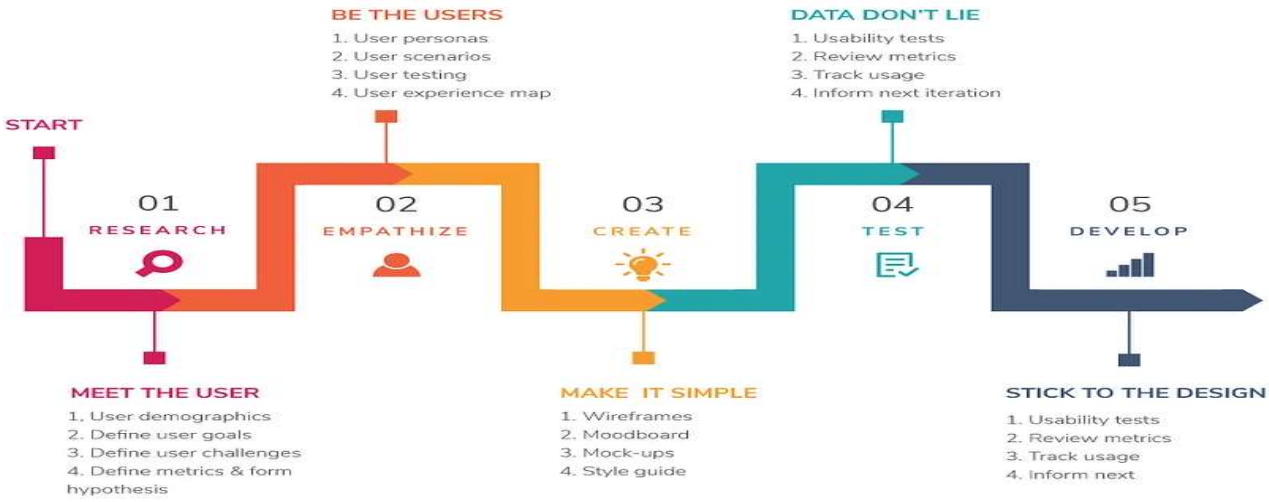
USER EXPERIENCE: Your app should give users a unique value, great usability, and good performance.

BUSINESS GOALS: Your app should have a clear vision along with marketing and financial logistics

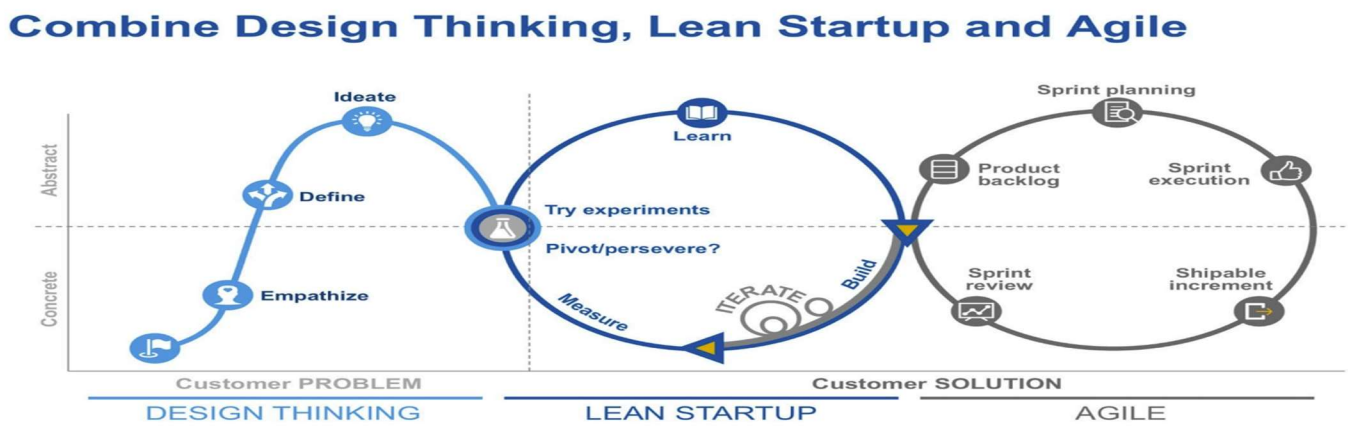
SECURITY & PRIVACY: Your app should handle user data and personal information safely, with the appropriate level of permission.

USER EXPERIENCE	BUSINESS GOALS
A great concept that solves a problem or fills a need. **A well-developed user interface** that is easy to navigate and pleasant to look at. **Solid functionality** that is easy to use but powerful enough to keep users coming back for more. **A marketing plan** will make users aware of the app and encourage them to download it. **A customer service plan** that makes it easy for current and potential customers to get in touch with you	**Increase** customer **engagement** and loyalty Boost **profits** **Improve working** condition **Balance** between **budget** and development **cost**

2. Mobile App Design Process



USER EXPERIENCE DESIGN PROCESS

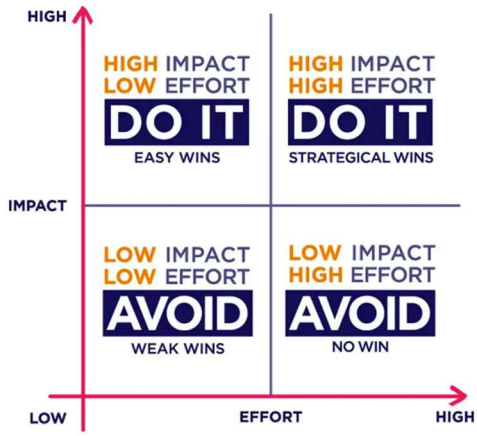


3. Idea Initiation: Research is a **key step** in designing the user experience

3.1 Human Interaction and Problem Solving	**3.2 Market Research**
What problems and pain points do I want to solve? How might I help users with that problem? What kind of app am I designing? (e.g., On Demand, Dating, Social) What is my app supposed to do? What is the unique selling point of my app? How is my app different from my competitors? How will my app appeal to my users? **HOW TO CREATE A BUSINESS CANVAS TEMPLATE For An App** IDEA: PROBLEM — What problem(s) are you solving? / SOLUTION — What is the solution to the problem? / VALUE — What value do you offer users? SELL: ADVANTAGE — What advantage do you have over competitors? / CUSTOMER/USER — Who is your ideal user? Where are they? / METRICS — What will you measure to mark your success? BUSINESS MODEL: CHANNELS — How will you reach and gain users? / COSTS — What are the costs of running your business? / REVENUE — How are you going to make money?	01 Survey your potential customers 02 Collect relevant data (Social Networks, Statistics,...) 03 Analyze your competitors ' SWOT 04 Apply results to your product **For Vino Veritas App** IDEA: PROBLEM — I don't know how to pick good wine. / SOLUTION — Pairings based on big data integration, matching algorithms, and expertise. / VALUE — Vino Veritas pairs users with wine based on palate, purpose, and price point. SELL: ADVANTAGE — • Advanced Matching Algorithm • Sommelier Experience / CUSTOMER/USER — • 21 + • Wine Amateurs • Young, Hip, Urban • Women / METRICS — • # of Subscribers • # of Downloads • User Retention • User Activity • Sales BUSINESS MODEL: CHANNELS — • Blog • Social Media • Events • Advertising • PR / COSTS — • App Design • App Development • Sales • Product Management • Marketing / REVENUE — • Sales via Subscription • Advertising • Events • Own Brand Product Sales

4. App Planning

Impact Effort Matrix **prioritize** your app's **valuable features**



MVP Features i**dentify required requirements**

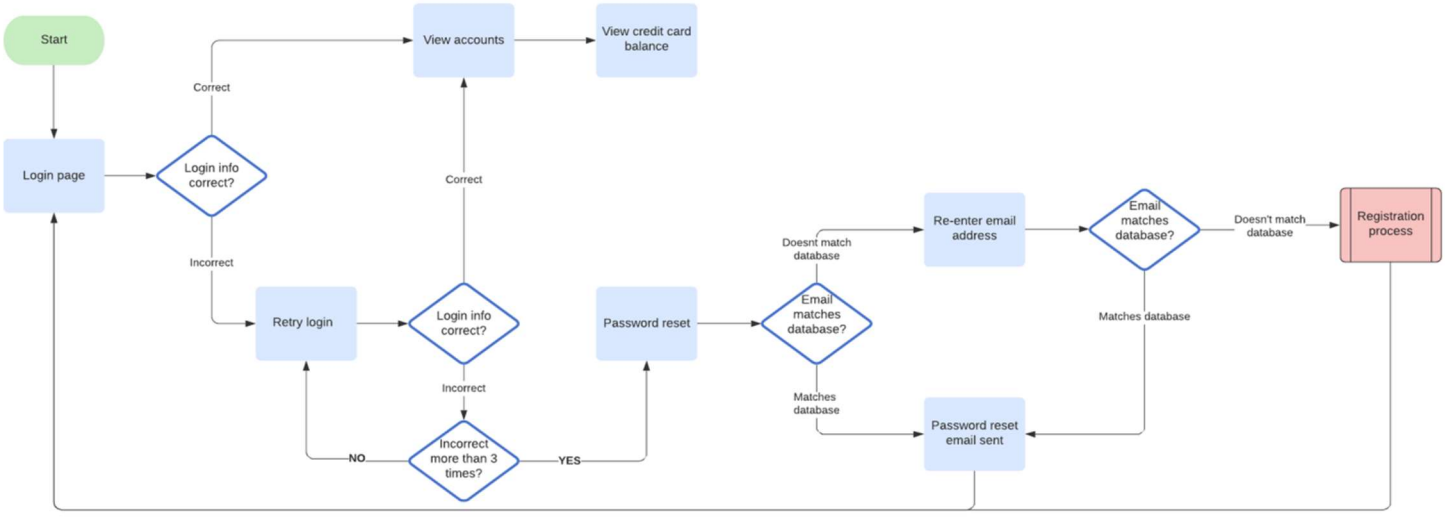
MUST HAVE FEATURES FOR MVP	SHOULD/ COULD HAVE FEATURES
Login/ Account Signup	Real Time Location Tracking
User Profile	Data Collection
Messaging	Data Pipeline
Push Notifications	Recommender System
Matching Algorithms	Loyalty Programs
Payment Integration	In-app Purchasing
Help and Support	Social Verification / Integration
Abuse Reporting / Monitoring	Reviews and Ratings

5. User story: A **general explanation** of a design **feature** from the **user's standpoint**

Structure: *As a [user], I want to/can [goal to achieve] to <receive value>."*

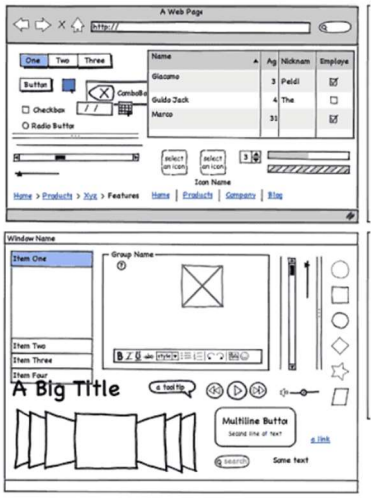
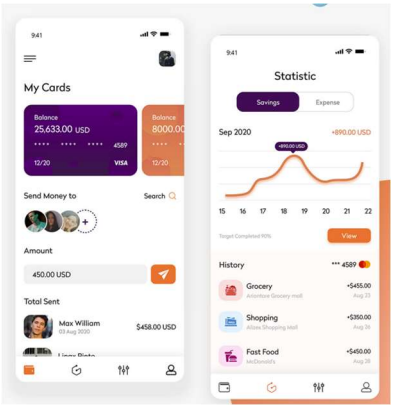
User Story	As a regular fintech app user, I want to withdraw cash from my bank account using the fintech mobile app, so that I can easily draw out cash even without using my card.
Acceptance Criteria	• User can login to the mobile app. • User can enter the desired amount to withdraw. • Desired amount is within the user's current balance. • Accounts are updated concurrently after validation. • Email and in-app confirmation of transactions are sent to the user.

5.1 User Flow Diagram



6. Wireframe, Mockup & Prototype

Why we need these things? To **Imagine** the app **correctly** & **Translate** your **ideas** into **visual presentations**

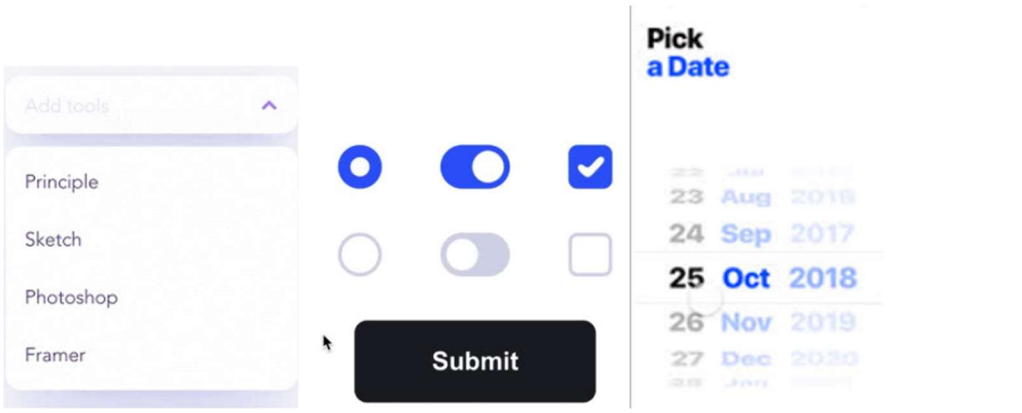
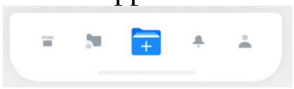
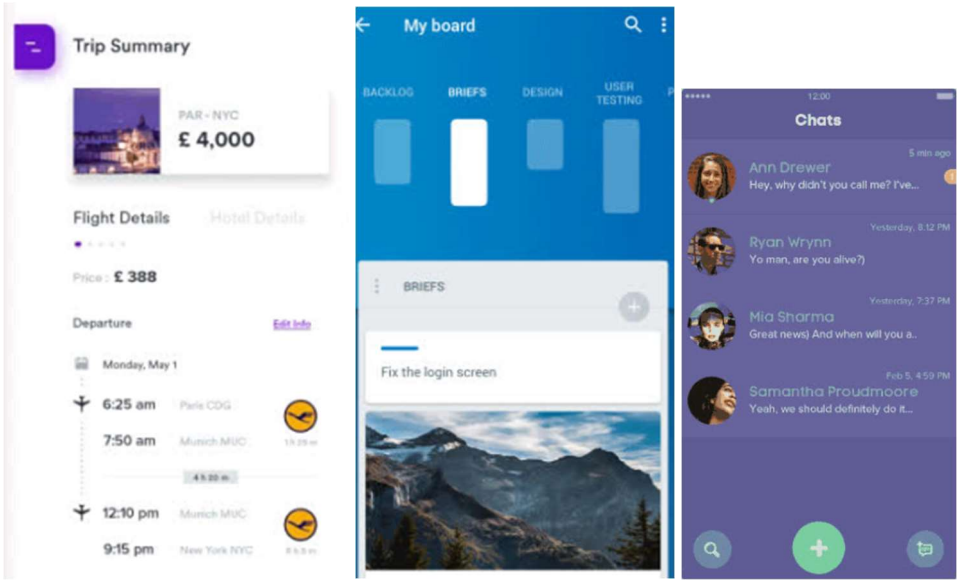
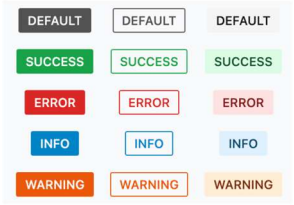
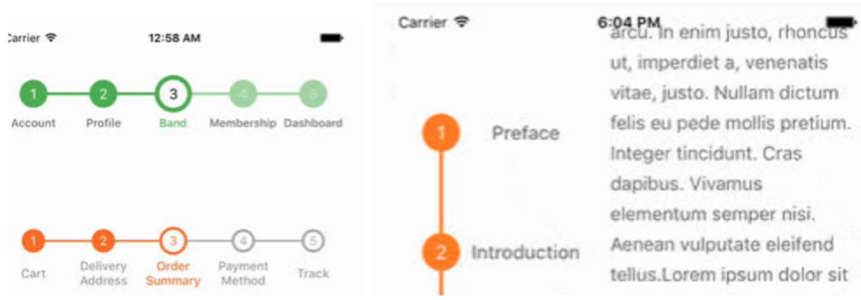
1. WIREFRAME	2. MOCKUP	3. PROTOTYPE
A wireframe is a basic, **low fidelity representation** of the initial product concept A **wireframe** traditionally only **uses black, white, and grey**, and can either be drawn by hand or created digitally	A mockup is **a static, high fidelity simulation** of the finished product that delivers the visual look of the product design—including typography, iconography, color, and overall style.	**Prototyping allows** designers to **test the user journey**: How the user might move between different actions or tasks to achieve certain outcomes
		

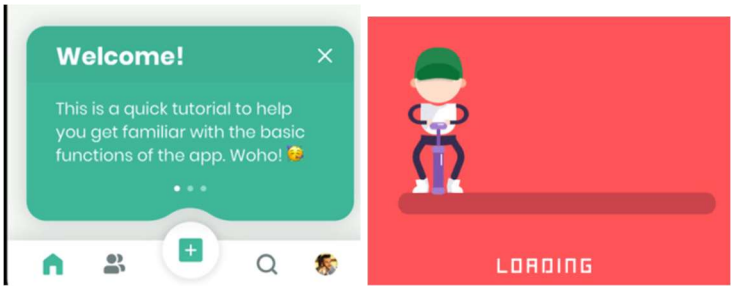
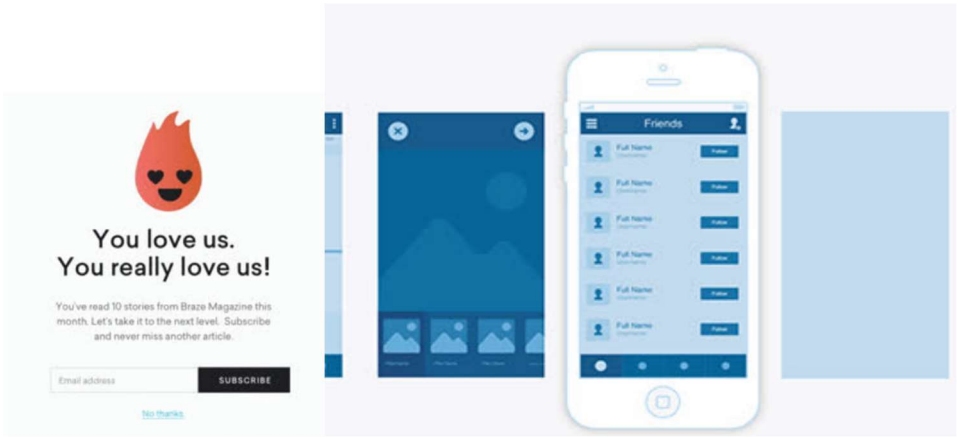
Ch3. Guide to Mobile App Design

1. User Interface Design Basics

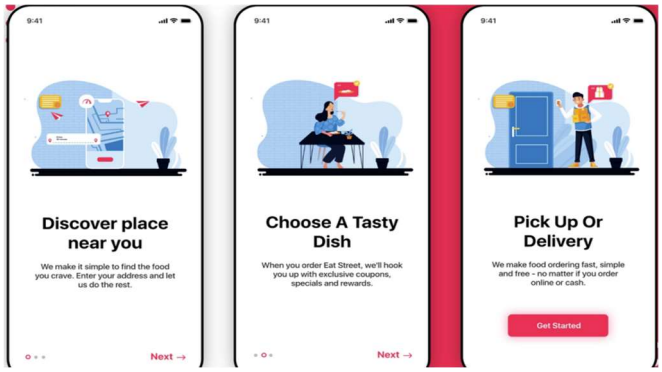
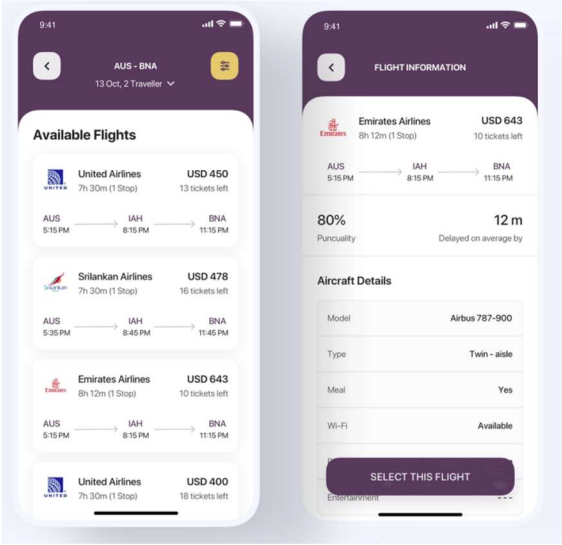
The **user interface (UI)** is the point at **which human users interact** with a computer, website, or mobile application.

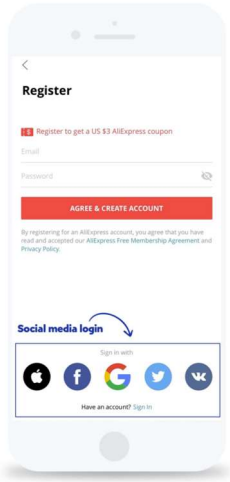
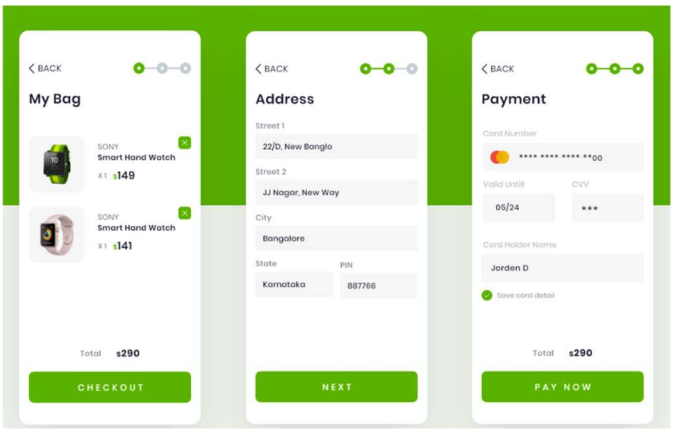
1.1 UI Elements

Input Controls	UI elements that enable users to input information Name_____ Address_____	
Navigation Components	UI components that can make it easy to navigate through a mobile app. 	
Information Components	UI components that help share information with users 	

		
Containers	UI components that can hold together a full set of components and content that are related to each other	

2. Mobile App Design Guidelines

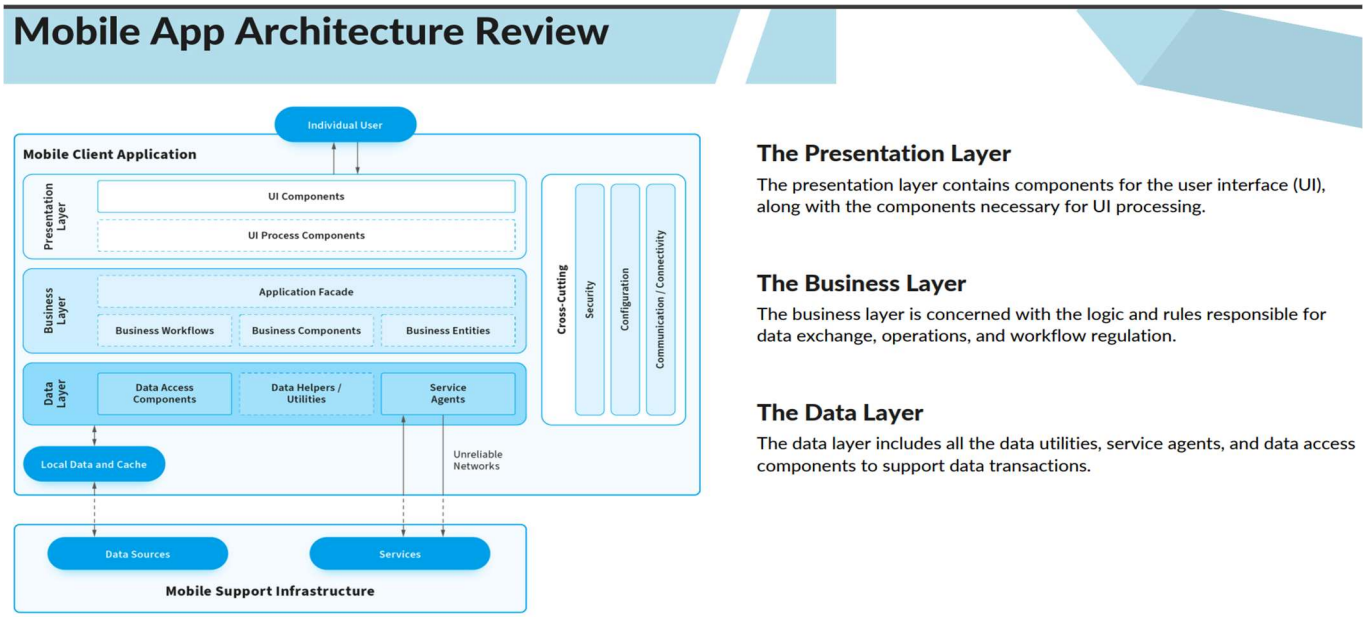
Make Onboarding Efficient	Show users the real value of your app. The very **first** thing a user will encounter is **onboarding**. Demonstrate how users can perform actions more smoothly and efficiently.	
Declutter	Remove all things that are not necessary for the app design Present the user with only what they need to know Keep interface elements to a minimum. A simple design will keep the user at ease with the product	

Minimize User Input	Keep forms as short as possible by removing any unnecessary fields • The app should ask for only the bare minimum of information from the user	
Breakdown Tasks	Divide large tasks into a number of subtasks. Do not create too much complexity for the user at one time.	

3. Best Practices and Tricks

Using autocomplete & value validate Use familiar screens Customize the keyboard for the type of query Use visual weight to convey the importance.	Avoid jargon Color-blindness Design for Fingers
Vector font Icons	Material Design Icon, Ionicons.
Vector Images	SVG
Vector Animation	Lottie.

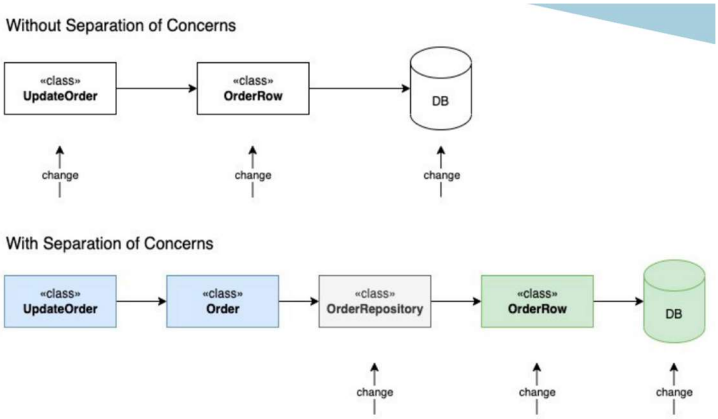
Ch4. Mobile Application Architecture

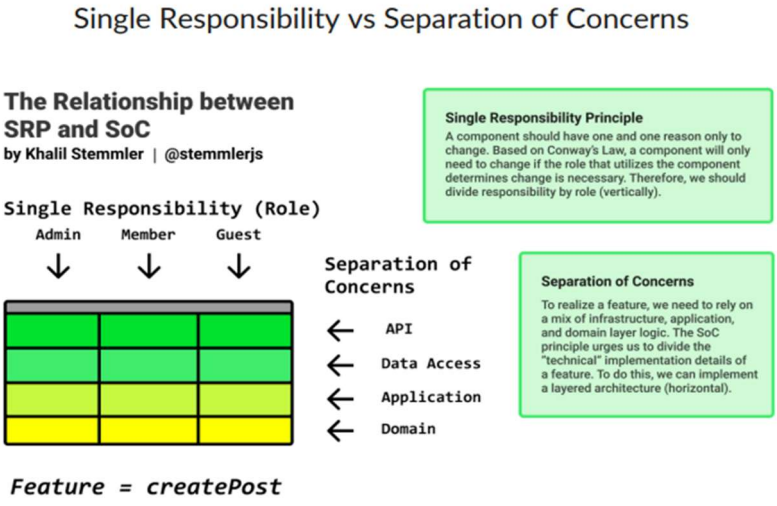


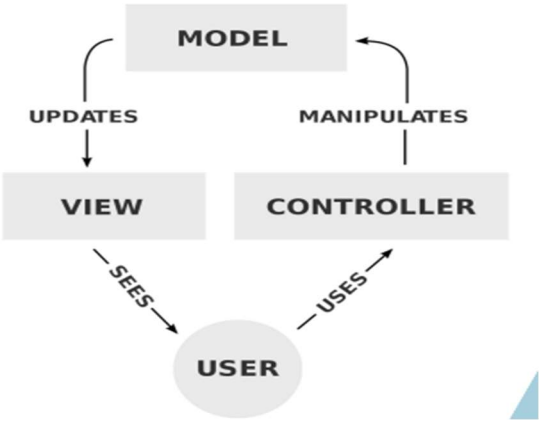
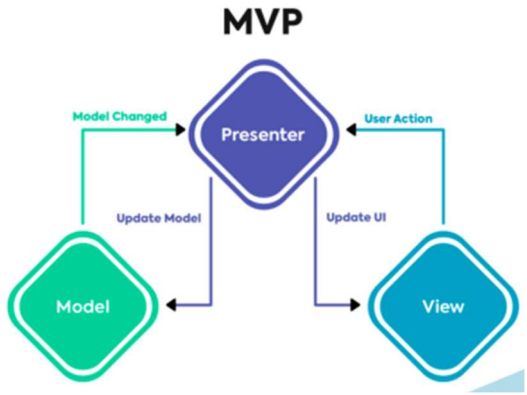
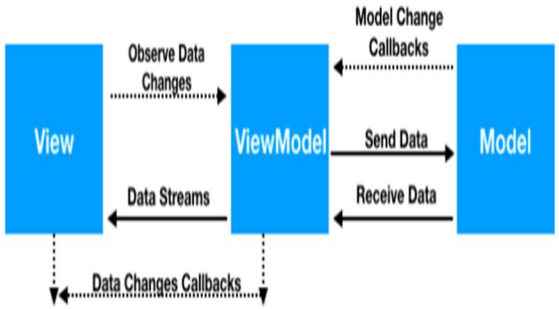
1. SEPARATION OF CONCERNS

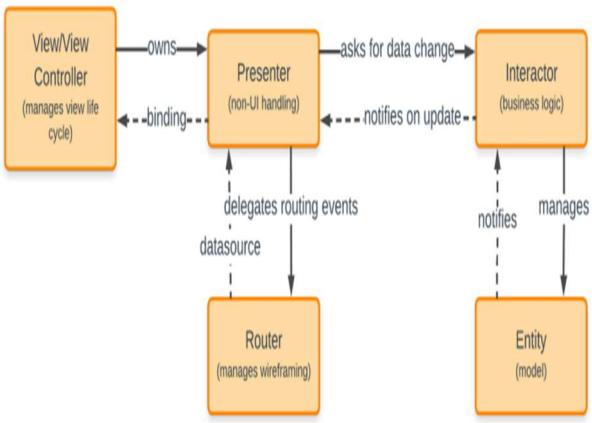
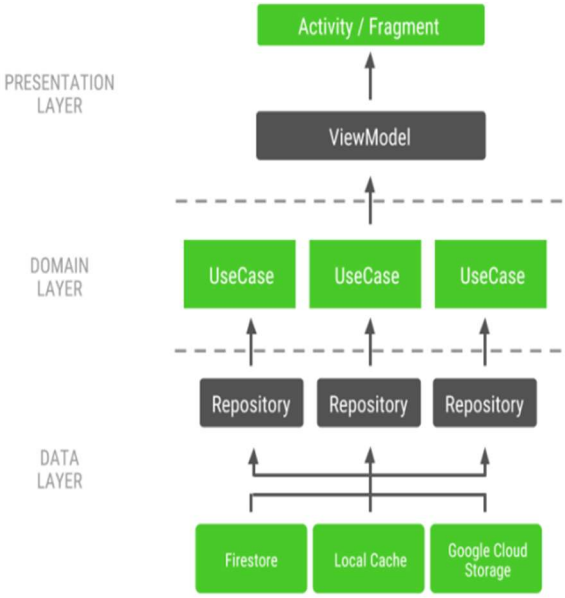
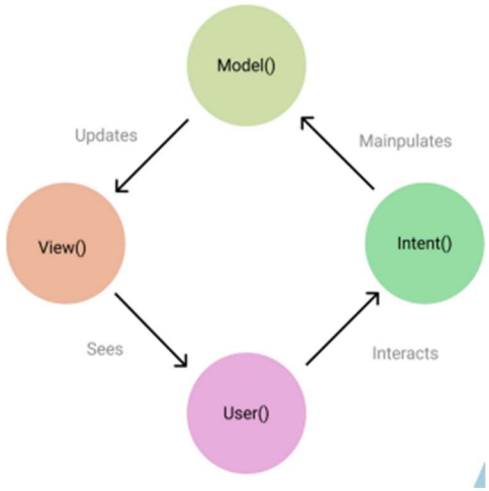
Separation of Concerns (SoC) is a design principle for separating a mobile app into distinct sections such that each section addresses a separate concern:

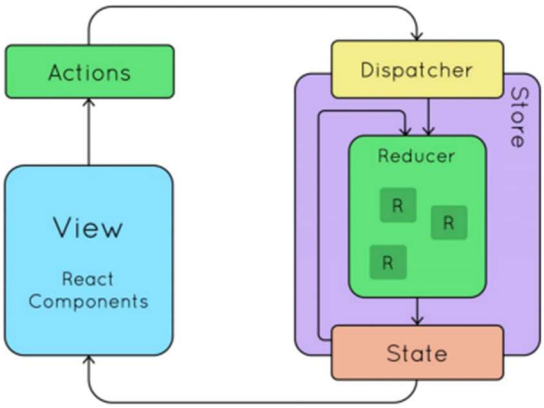
- These layers should be loosely coupled
- Changes in one layer having limited or no impact on any other layers

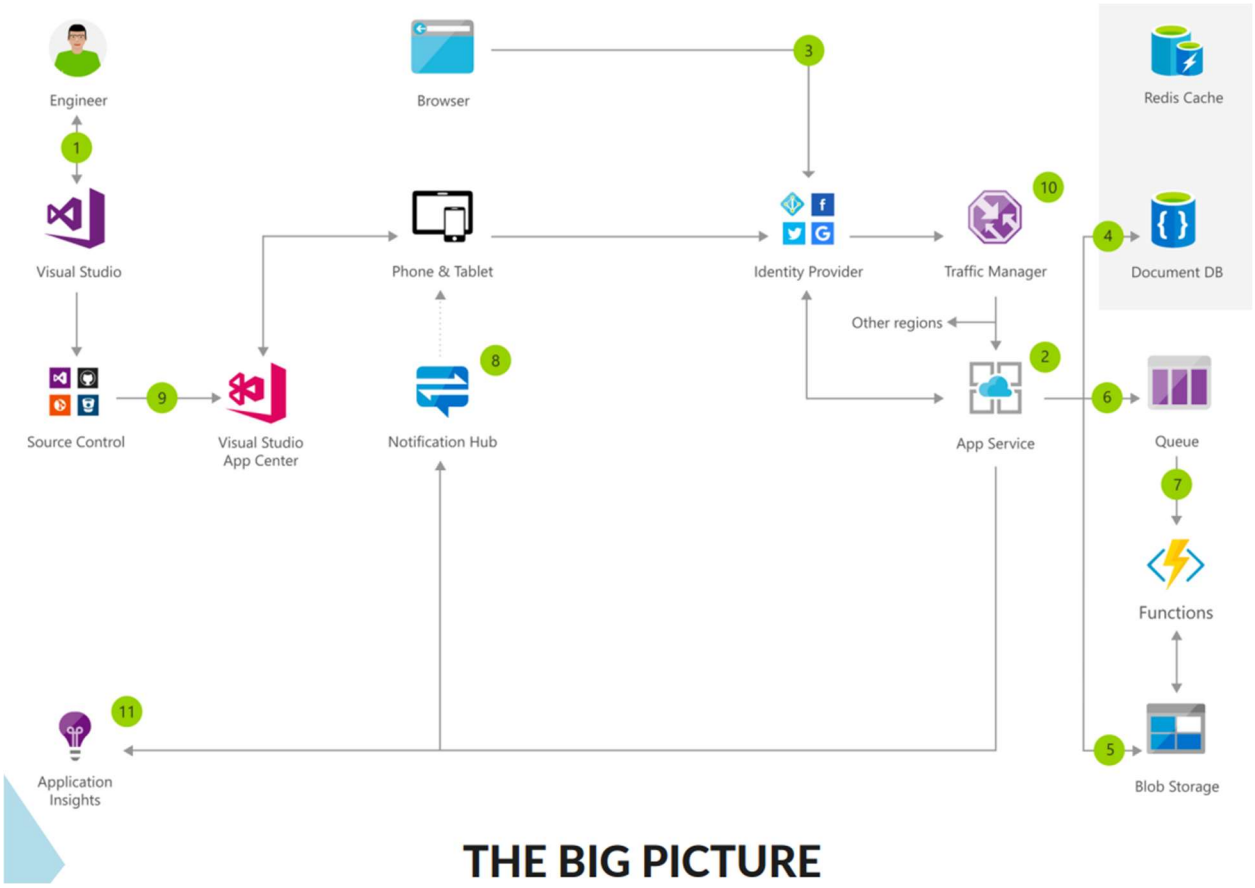




Model View Controller	- The **model** is responsible for managing the data of the application. It receives user input from the controller. - The **view** renders a presentation of the model in a particular format. - The **controller** responds to the user input and performs interactions on the data model objects. - *The controller receives the input, optionally validates it, and then passes the input to the model*	
Model View Presenter *In MVP, the role of the* ***controller*** *is* ***replaced*** *with a* ***Presenter****.*	- The **Model** represents a set of classes that describes the business logic and data. It also defines business rules for data means how the data can be changed and manipulated. - The **View** represents the UI components. It is only responsible for displaying the data that is received from the presenter as the result. This also transforms the model(s) into UI. - The **Presenter** is responsible for handling all UI events on behalf of the view. This receives input from users via the View, then process the user's data with the help of the Model and passes the results back to the View	
Model View ViewModel *This pattern supports two-way data binding between view and View model*	**- Model**: This holds the data of the application. It cannot directly talk to the View. Generally, it's recommended to expose the data to the ViewModel through Observables. **- View**: It represents the UI of the application devoid of any Application Logic. It observes the ViewModel. **- ViewModel**: It acts as a link between the Model and the View. It's responsible for transforming the data from the Model. It provides data streams to the View. It also uses hooks or callbacks to update the View. It'll ask for the data from the Model.	

VIPER (iOS)	**- View (Controller)**: this presents UI to the user and forwards the actions on UI to the Presenter to handle. **- Presenter**: handles the UI actions sent by View. This handling is totally independent of the UI controls. In addition to this, Presenter takes the help of the Interactor to process the model(Entity) if need be. **- Interactor**: mainly contains the business logic and integrates it with the other parts of the application. It also owns and manages the Entity. **- Entity**: is just a dumb data structure to store data. As said, it is owned by the Interactor and concealed from the other modules. **- Router**: is a special component in the VIPER architecture, which explicitly deals with the navigation actions coming from the Presenter. Navigation (Wireframing) is the only responsibility that the Router takes care of.	
Android App Architecture *Based on Clean Architecture*	**- Presentation layer**: The role of the UI layer (or presentation layer) is to display the application data on the screen. Whenever the data changes, the UI should update to reflect the changes **- Domain layer**: The domain layer is responsible for encapsulating complex business logic, or simple business logic that is reused by multiple ViewModels. **- Data layer**: The data layer of an app contains the business logic. The business logic is what gives value to your app—it's made of rules that determine how your app creates, stores, and changes data	
Model View Intent *MVI imagines the user as a function, and models a unidirection-al data flow based on that.*	**- Model**: Other than representing the data and the business logic of the application model maintains a state which can be changed through the actions of the user. **- View**: Represents the UI layer of the application which consists of Activities and Fragments. It accepts different model states and displays them as a UI. **- Intent**: do not confuse it with the Android Intent, it's the intention to perform an action either by the user or the app itself	

React Redux	**- Store:** This is simply a state container that holds on to your app state. It holds an immutable reference representing the entire state of your application and can only be updated by dispatching an Action to it. **- Action:** Actions contain information to be sent to the store. They represent how we want our state to be changed **- Reducers:** Since neither Actions nor the Store update state themselves, we need a component dedicated to doing this. That's where Reducers come in. They simply take an Action and State and emit a new State.	



II. Ôn tập

1.222

[222] Đối với kiến trúc MVI, Intern được định nghĩa là? Mong muốn của người dùng khi thực thi một hành động.

Ch5. Web Services

1.WEB SERVICES

1.1 What is a web service ?

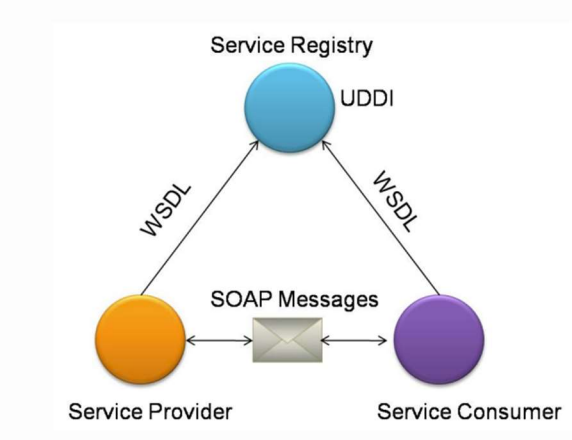
- Is available over the Internet or private (intranet) networks
- Uses a standardized XML messaging system
- Is not tied to any one operating system or programming language
- Is self-describing via a common XML grammar
- Is discoverable via a simple find mechanism

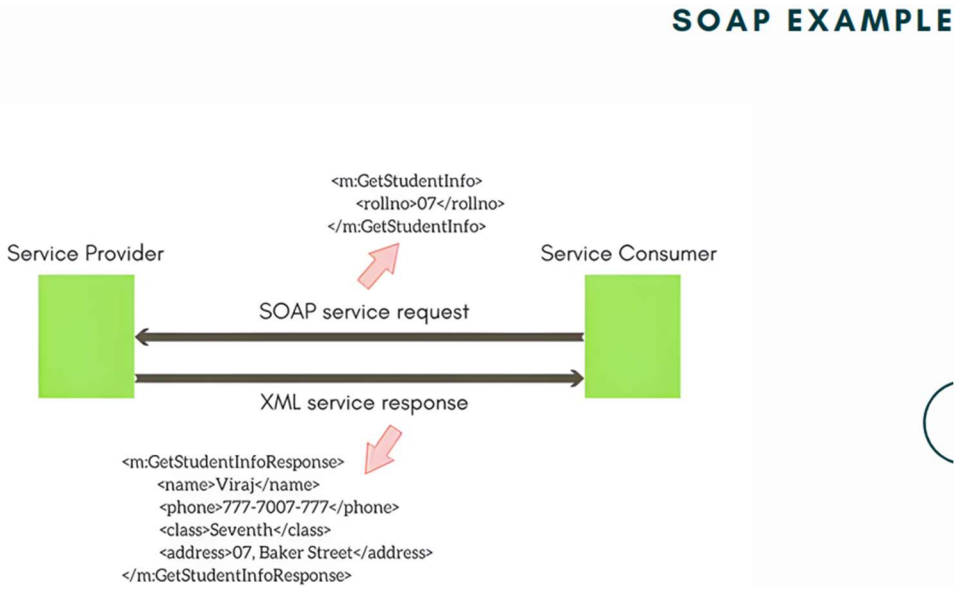
1.2 WEB SERVICE COMPONENTS

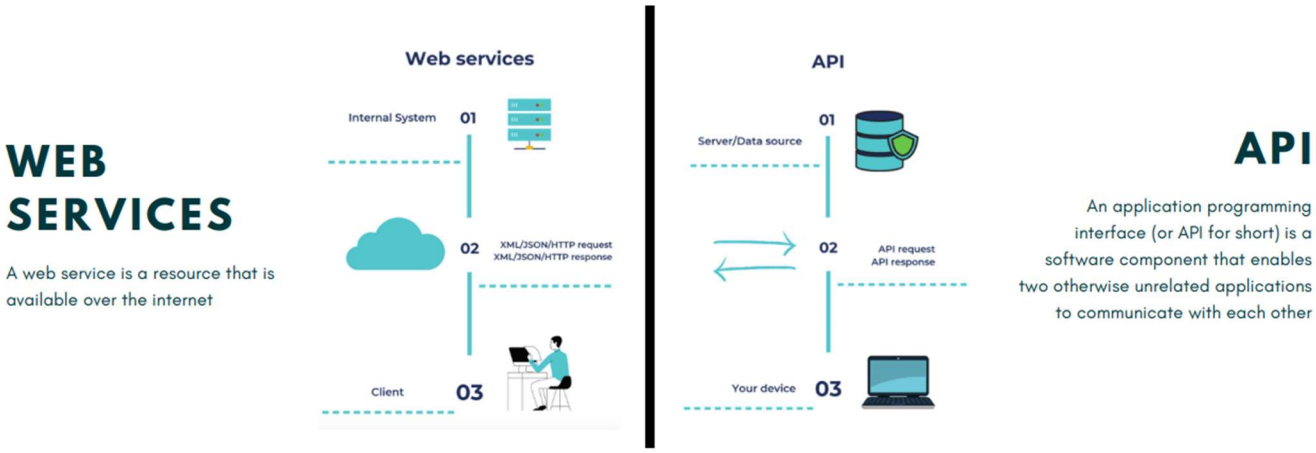
- SOAP (Simple Object Access Protocol): SOAP is an XML-based protocol for accessing web services

- UDDI (Universal Description, Discovery and Integration): UDDI is an XML based framework for describing, discovering, and integrating web services.

- WSDL (Web Services Description Language). WSDL is an XML document containing information about web services such as method name, method parameter, and how to access it.



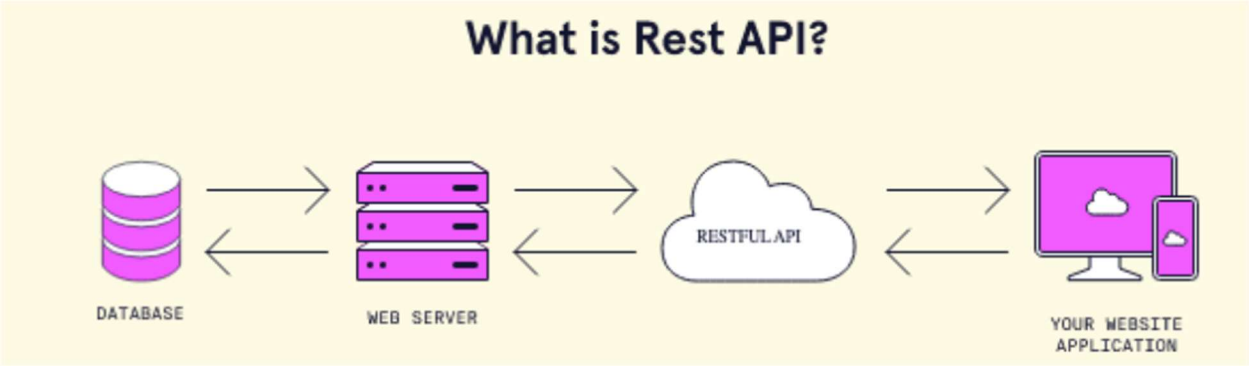




2. INTRODUCTION TO REST

2.1 What is REST *(REpresentational State Transfer)*

REST is an **architectural style** for providing standards between **computer systems** on the **web**



2.2 REST Methods

GET *(READ)*	Use GET requests to only retrieve remote resources Additionally, GET APIs should be **idempotent**. Making multiple identical requests must produce the same result every time until another API (POST or PUT) has changed the state of the resource on the server. **Response**: - For any given HTTP GET API, if the resource is found on the server, then it must return HTTP response code 200 (OK) - In case the resource is NOT found on the server then API must return HTTP response code 404 (NOT FOUND). - Similarly, if it is determined that the GET request itself is not correctly formed then the server will return the HTTP response code 400 (BAD REQUEST).	- HTTP GET: http://www.appdomain.com/users - HTTP GET: http://www.appdomain.com/users?size=20&page=5 - HTTP GET: http://www.appdomain.com/users/123 - HTTP GET: http://www.appdomain.com/users/123/address
POST *(CREATE)*	Use POST APIs to create new subordinate resources, e.g., a file is subordinate to a directory containing it or a row is subordinate to a database table. Response: - Ideally, if a resource has been created on the origin server, the response SHOULD be HTTP response code 201 (Created) and	● HTTP POST http://www.appdomain.com/users ● HTTP POST http://www.appdomain.com/users/123/accounts

	contain an entity that describes the status of the request and refers to the new resource, and a Location header. - Many times, the action performed by the POST method might not result in a resource that can be identified by a URI. In this case, either HTTP response code 200 (OK) or 204 (No Content) is the appropriate response status.	
PUT *(UPDATE /REPLACE)*	Use PUT APIs primarily to update an existing resource (if the resource does not exist, then API may decide to create a new resource or not). **Response Codes** - If a new resource has been created by the PUT API, the origin server MUST inform the user agent via the HTTP response code 201 (Created) response. - If an existing resource is modified, either the 200 (OK) or 204 (No Content) response codes SHOULD be sent to indicate successful completion of the request. *The difference between the POST and PUT APIs can be observed in request URIs. POST requests are made on resource collections, whereas PUT requests are made on a single resource.*	● HTTP PUT: http://www.appdomain.com/users/123 ● HTTP PUT: http://www.appdomain.com/users/123/accounts/456
DELETE *(DELETE)*	As the name applies, DELETE APIs delete the resources (identified by the Request-URI). DELETE operations are **idempotent**. If you DELETE a resource, it's removed from the collection of resources. If the request passes through a cache and the Request-URI identifies one or more currently cached entities, those entries SHOULD be treated as stale. Responses to this method are not cacheable. **Response Codes:** - A successful response of DELETE requests SHOULD be an HTTP response code 200 (OK) if the response includes an entity describing the status. - The status should be 202 (Accepted) if the action has been queued. - The status should be 204 (No Content) if the action has been performed but the response does not include an entity. - Repeatedly calling DELETE API on that resource will not change the outcome – however, calling DELETE on a resource a second time will return a 404 (NOT FOUND) since it was already removed	HTTP DELETE: http://www.appdomain.com/users/123 HTTP DELETE: http://www.appdomain.com/users/123/accounts/456
PATCH *(Partial Update/Modify)*	● HTTP PATCH requests are to make a partial update on a resource. ● If you see PUT requests modify a resource entity too. So to make it more precise – the PATCH method is the correct choice for partially updating an existing resource, and you should only use PUT if you're replacing a resource in its entirety.	HTTP PATCH /users/1 [{ "op": "replace", "path": "/email", "value": "new.email@example.org" }]

2.3 REST Resources

- A resource can be a **singleton** or a **collection**.
    - Ex: customers is a collection resource and customer is a singleton resource (in a banking domain).
- We can **identify the customers collection resource** using the **URI "/customers ".** We can identify a single " customer " resource using the URI "/customers/{customerId}".
- A resource may contain sub-collection resources also.
    - Ex: sub-collection resource "accounts" of a particular " customer " can be identified using the URN "/customers/{customerId}/accounts "

2.4 RESTful Guidelines

- **CLIENT-SERVER** Client applications and server applications MUST be able to evolve separately without any dependency on each other.
- **STATELESS** The server will not store anything about the latest HTTP request the client made. It will treat every request as new. No session, no history.
- **UNIFORM INTERFACE** All resources should be accessible through a common approach such as HTTP GET and similarly modified using a consistent approach.
- **CACHEABLE** Caching shall be applied to resources when applicable, and then these resources MUST declare themselves cacheable. Caching can be implemented on the server or client-side.
- **LAYERED SYSTEM** Each layer doesn 't know anything about any layer other than that of immediate layer and there can be lot of intermediate servers between client and the end server.
- **CODE ON DEMAND (OPTIONAL)** REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts

3. SOAP vs REST

SOAP	REST
- SOAP is a protocol - SOAP cannot make use of REST - SOAP uses service interfaces to expose its functionality to client applications - SOAP requires more bandwidth for its presentation - SOAP can only work with XML format. As seen from SOAP messages, all data passed is in XML format. - Support Stateful operations and security mechanisms	- REST is an architectural style - REST can make use of SOAP - REST uses Uniform Service locators to access the components on the hardware device. - REST messages is lightweight - REST permits different data formats such as Plain text, HTML, XML, JSON, etc. - Support caching

III. DESIGN A REST API

1. Idempotent REST APIs

- Idempotency is the ability for an operation to be carried out multiple times without changing the state of the system.
- An idempotent HTTP method is a method that can be invoked many times without different outcomes.
- It should not matter if the method has been called only once, or ten times over. The result should always be the same.



2. REST Best practices

Use nouns to represent resources RESTful URI should refer to a resource that is a thing (noun) instead of referring to an action (verb) because nouns have properties that verbs do not have – similarly, resources have attributes

- *http://api.example.com/device-management/managed-devices*
- *http://api.example.com/device-management/managed-devices/{device-id}*
- *http://api.example.com/user-management/users*

Use Nesting on Endpoints to Show Relationships

- Ex: A blog post might have their comments, so to retrieve these comments, an endpoint like
- *https://mysite.com/posts/{postId}/comments* would make sense

Use Nouns Instead of Verbs in Endpoints

- Because HTTP methods such as GET, POST, PUT, PATCH, and DELETE are already in verb form for performing basic CRUD (Create, Read, Update, Delete) operations, the endpoints should use nouns, signifying what each of them does.

Use JSON as the Format for Sending and Receiving Data

- To ensure the client interprets JSON data correctly, you should set the Content-Type type in the response header to application/json while making the request

Use Filtering, Sorting, and Pagination to Retrieve the Data Requested

GET requests over collection resources can potentially return a large number of items. You should design a web API to limit the amount of data returned by any single request. *Ex: /orders?limit=25&offset=50*

Be Clear with Versioning

REST APIs should have different versions, so you don ’t force clients (users) to migrate to new versions. This might even break the application if you ' re not careful.

Many RESTful APIs from tech giants and individuals usually comes like this:

- *https://mysite.com/v1/ for version 1*
- *https://mysite.com/v2 for version 2*

Handle errors gracefully and return standard error codes

To eliminate confusion for API users when an error occurs, we should handle errors gracefully and return HTTP response codes that indicate what kind of error occurred
Common error HTTP status codes include:
- 400 Bad Request – This means that client-side input fails validation.
- 401 Unauthorized – This means the user isn ’t not authorized to access a resource. It usually returns when the user isn ’t authenticated.
- 403 Forbidden – This means the user is authenticated, but it’ s not allowed to access a resource.
- 404 Not Found – This indicates that a resource is not found.
- 500 Internal server error – This is a generic server error. It probably shouldn ’t be thrown explicitly.
- 502 Bad Gateway – This indicates an invalid response from an upstream server.
- 503 Service Unavailable – This indicates that something unexpected happened on server side (It can be anything like server overload, some parts of the system failed, etc.).

Cache data to improve performance

- We can add caching to return data from the local memory cache instead of querying the database to get the data every time we want to retrieve some data that users request. The good thing about caching is that users can get data faster. However, the data that users get may be outdated.

Provide Accurate API Documentation

- When you make a REST API, you need to help clients (consumers) learn and figure out how to use it correctly. The best way to do this is by providing good documentation for the API.
- One of the most common tools you can use for API documentation is Swagger. And you can also use Postman, one of the most common API testing tools in software development, to document your APIs

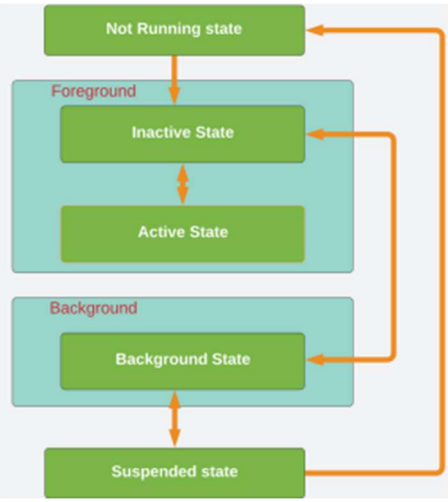
Ch. 6 Mobile Application Life Cycle.

1. Mobile App Life Cycle

Handling Life Cycle in Mobile App is **important**!

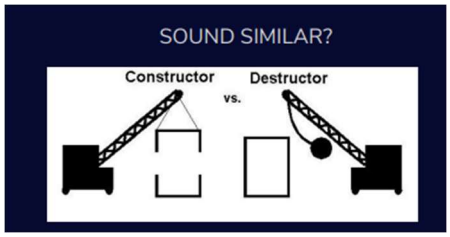
Recall the difference between Mobile Apps and other platforms:

- Activity duration.
- Hardware resources.
- Responsiveness.
- System events ...



2. Life cycle callback:

Allow us to know that a state has changed: System is creating, stopping or resuming our apps, or destroying the process.

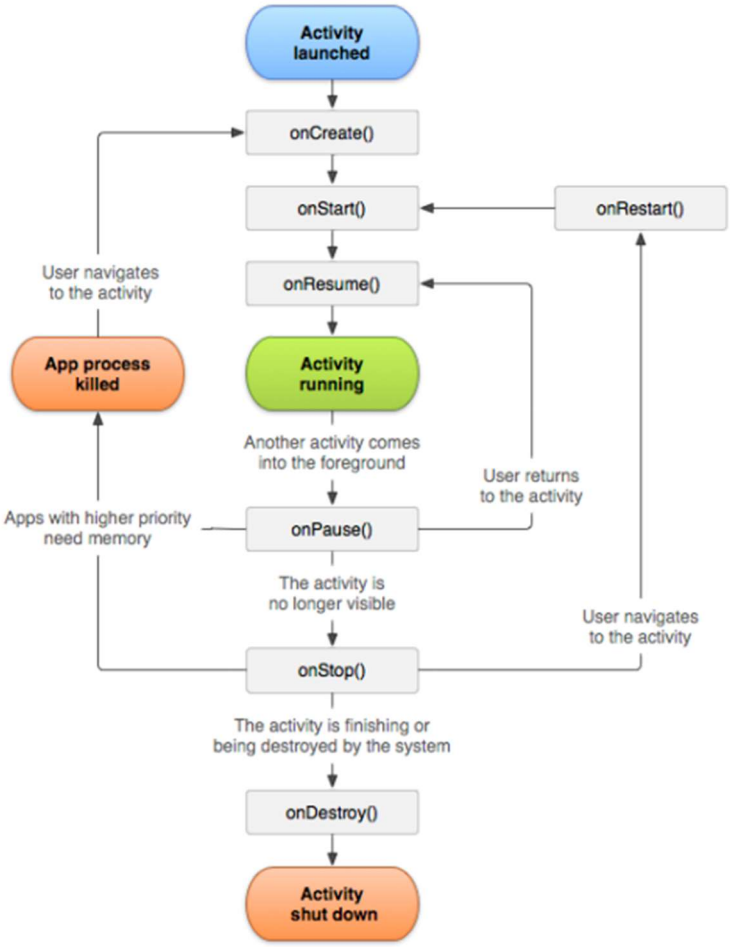
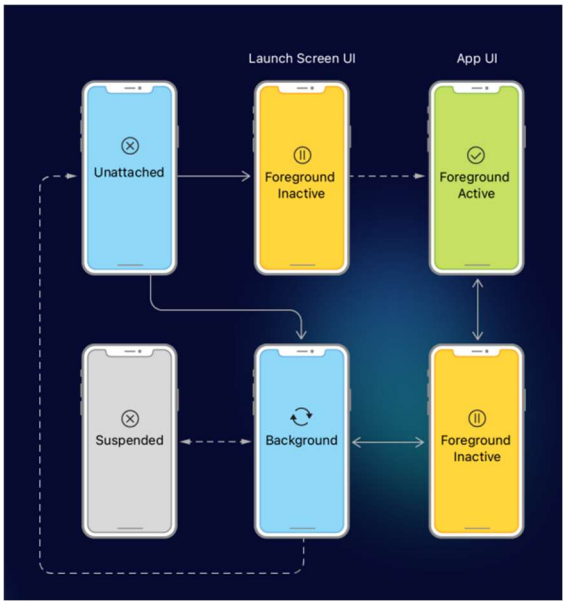


3. Key to any robust and efficient mobile appps.

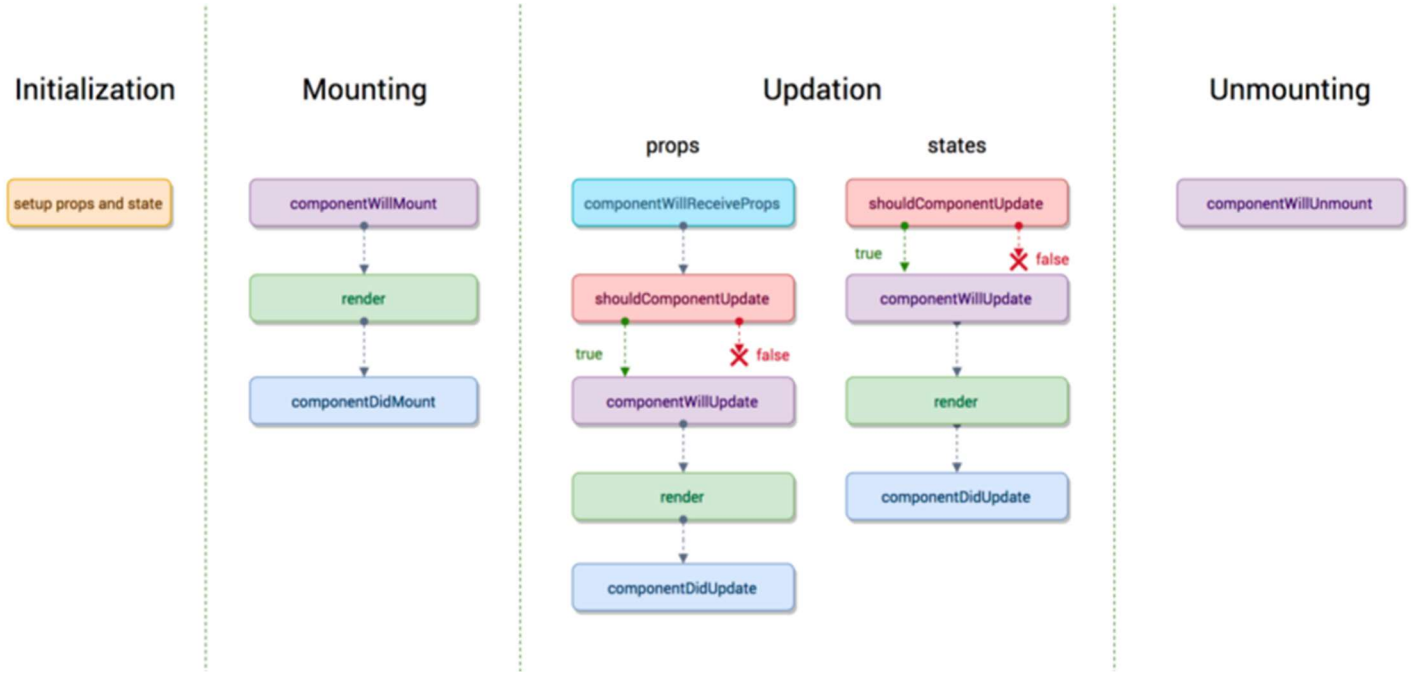
A good implementation of the lifecycle callbacks can help ensure that your app avoids:

- Crashing when receive call or switch to other apps.
- Consuming valuable system resources when user not **actively** use.
- Lose progress when user leaves your apps and returns at a later time.
- Crash / lose progress when rotates screen.

II. Android & iOS App Life Cycle

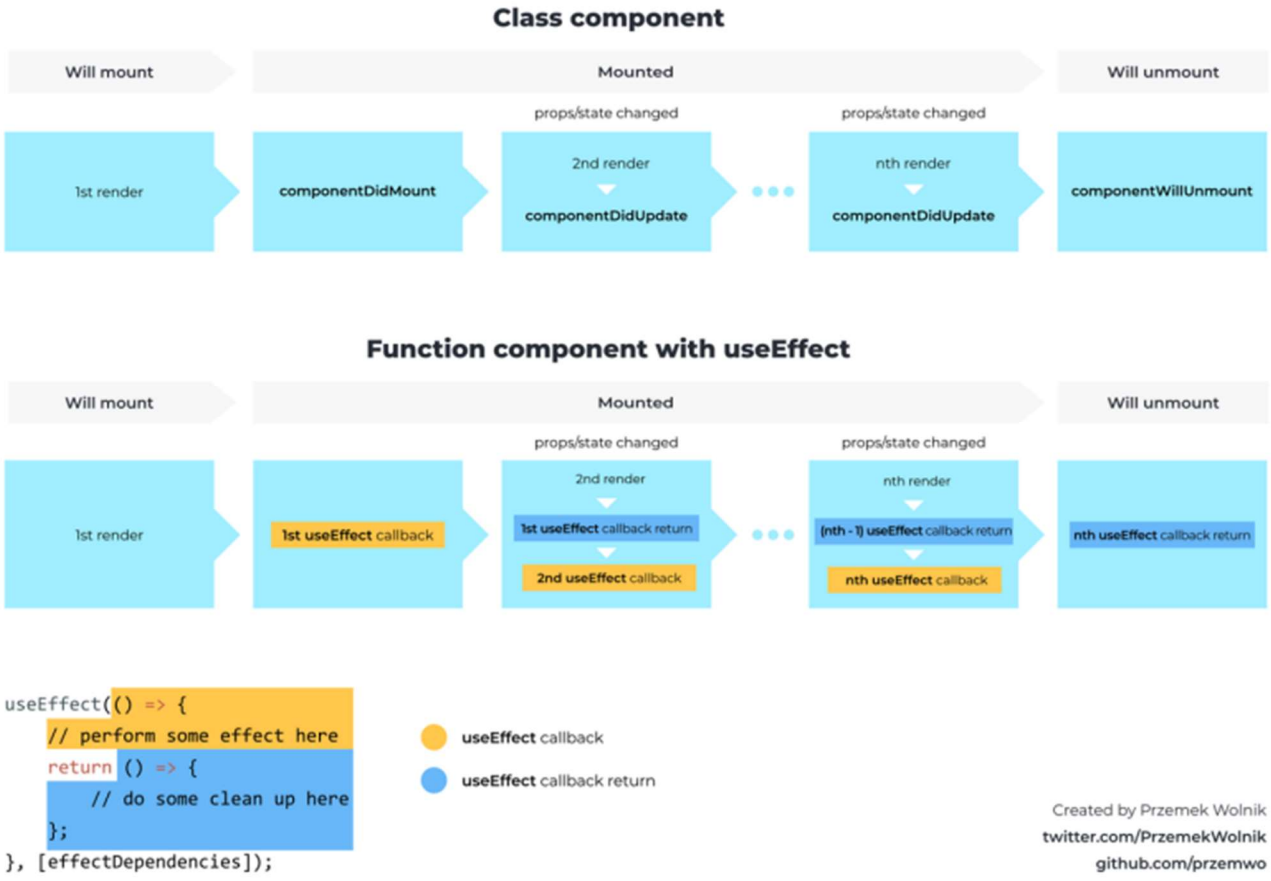
Android Activity Life Cycle	As a user navigates through, out of, and back to our app, the Activity instances transition through different states in their lifecycle.	
iOS Scene Life Cycle	The introduction of multi-window support on iOS 13 (iPadOS) brings major changes in application's life cycle	

React class component life cycle.



- Không chỉ có chuyện đóng/mở apps mà còn liên quan đến việc cập nhật trạng thái trên apps.

React Function Component Life Cycle with Hooks

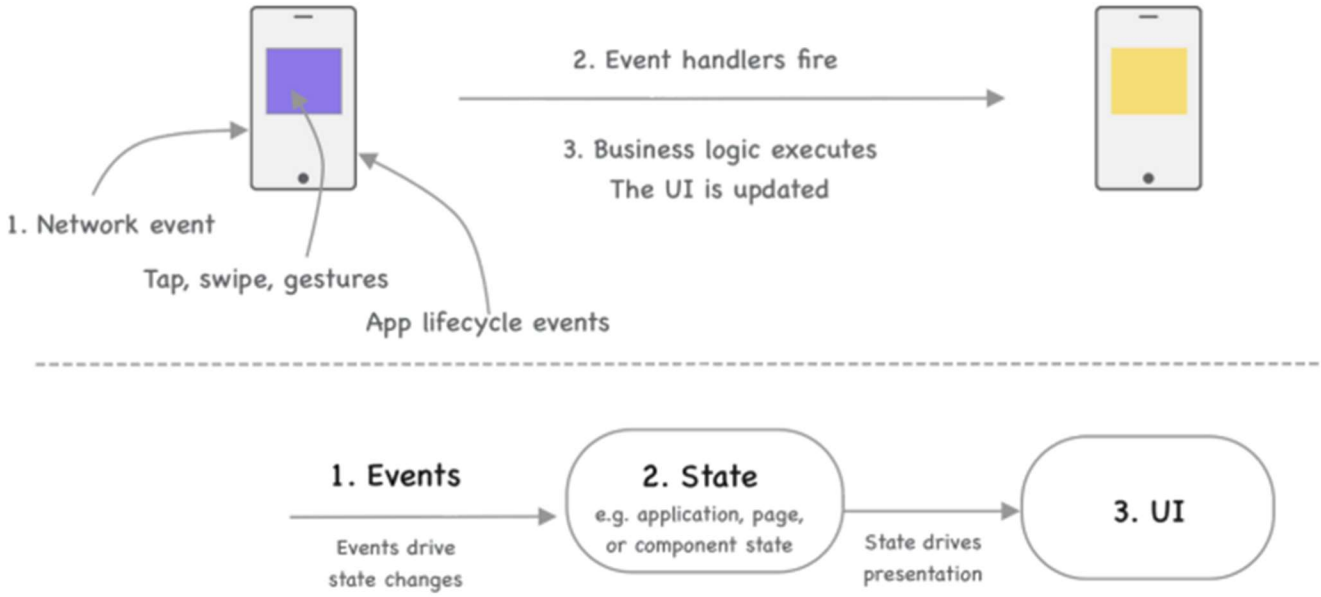


III. Mobile App State management.

1. State management: is the process of

- Maintain the knowledge of an application's UI controls across multiple related data flows that form a complete business use case.

- Understand the condition of the app at any given moment.



2. Type of App State: The state of an app is everything that exists in memory when the app is running

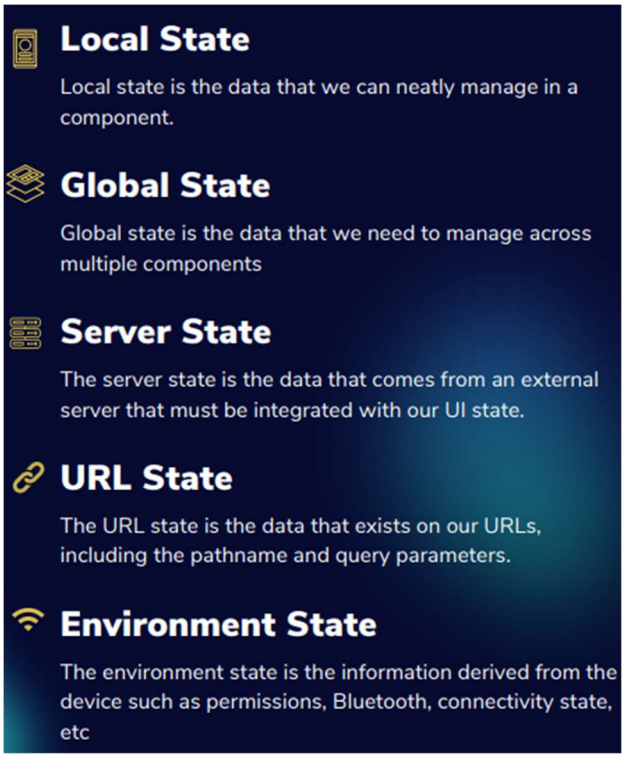
Local State: Thông tin có giá trị trên màn hình đó.
VD: Thông tin hiển thị tính toán trên màn hình

Global State Dữ liệu chia sẻ giữa các màn hình với nhau. VD: Token của người dung, thông tin cá nhân. Ngôn ngữ sử dụng.

Server state: Lưu thông tin từ Web Service về.

URL State: Query dữ liệu như thế nào.

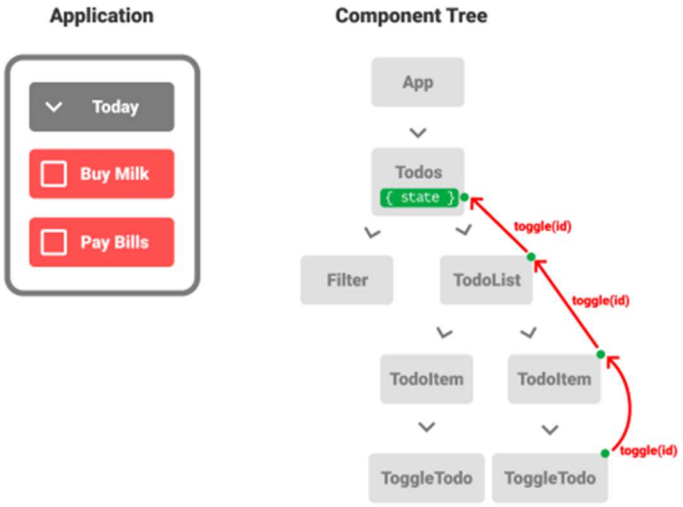
Environment State: Có cảm biến nào thì lưu tại đây.



IV. Redux overview.

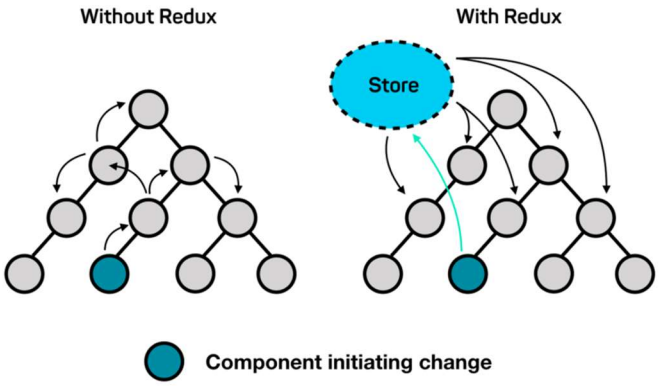
1. Prop drilling problem.

Prop drilling is the problem for passing data through several nested children components, in a bid to deliver this data to a deeply-nested component.



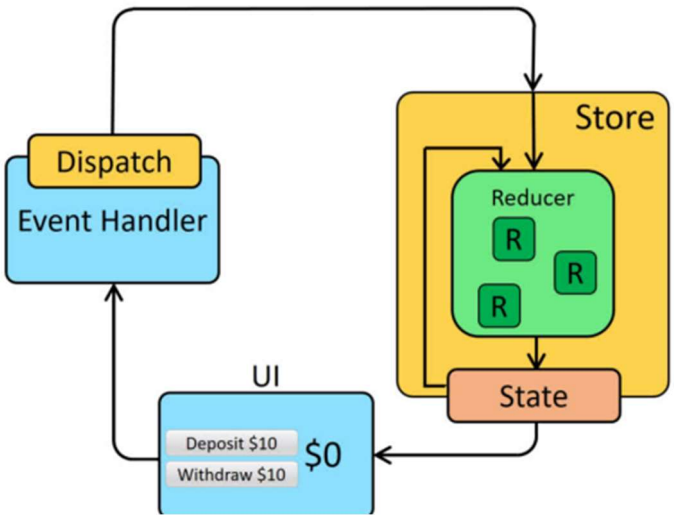
2 **Manage shared state with Redux.**

- Giúp cho những cái gì chia sẻ mất thời gian thì chúng ta sẽ có nơi tập trung.
- Vấn đề: App chậm hon.
- Những app nhỏ áp dụng redux vào thì sẽ hơi nặng khi chưa biết cách quản lý.
- Không làm kĩ: Gây quá tải chp phần render bên dưới.



3. Redux is Event Driven.

- Actions are dispatched in response to an event like a UI click
- The store runs the reducer function to calculate a new state based on the event –
- The UI reads the new state to display the new values



Ví dụ:

- App ngân hàng: Số dư là biến toàn cục là globa, State.

- Khi thay đổi tiền, hành động đó sẽ đc đưa đến trung tâm để xứ lý hành động đó.

- Nên sử dụng React tool kit để làm nhanh hơn.

- Kết hợp nó vào cái luồng để gọi API.

- Có những cái app mượn, có những cái app lag ? Quản lý trạng thái.

- Những apps như fb, ta vẫn thấy dữ liệu từ lần cuối mình xài. Đó là cơ chế lưu giữ trạng thái. Khi mà có mạng sẽ update cái mới.

B. React Native.

- Không phải ai cũng xài kiến trúc mới do thư viện ko hỗ trợ.

- Expo: App native được build sẵn, cho phép thay thể JS builder trong đó.

- Vấn đề:

+ ko thể tối ưu hoá.

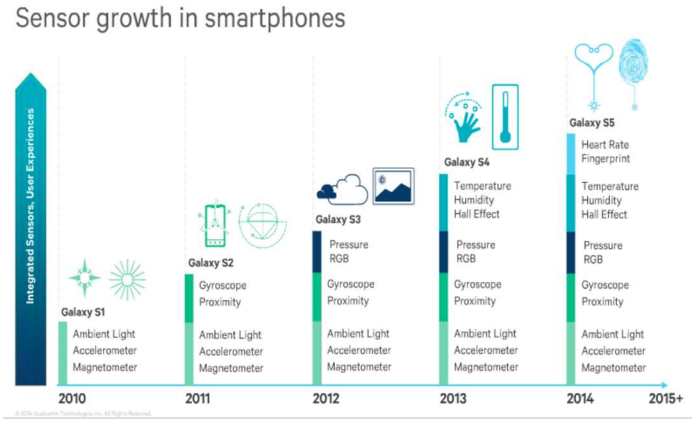
+ giới hạn mới các modules đc cung cấp sẵn.

+ Passively what for the core react-native upgrade from expo dev team.

CHAPTER 07 MOBILE DEVICE SENSORS

1. INTRODUCTION TO MOBILE DEVICE SENSORS

A **sensor** is a tiny device or module that analyzes its surroundings and reports the quantitative measure to the controlling program. Smartphones today come with a wealth of sensors to facilitate a better user experience, and provide apps with enhanced information about the world around the use.



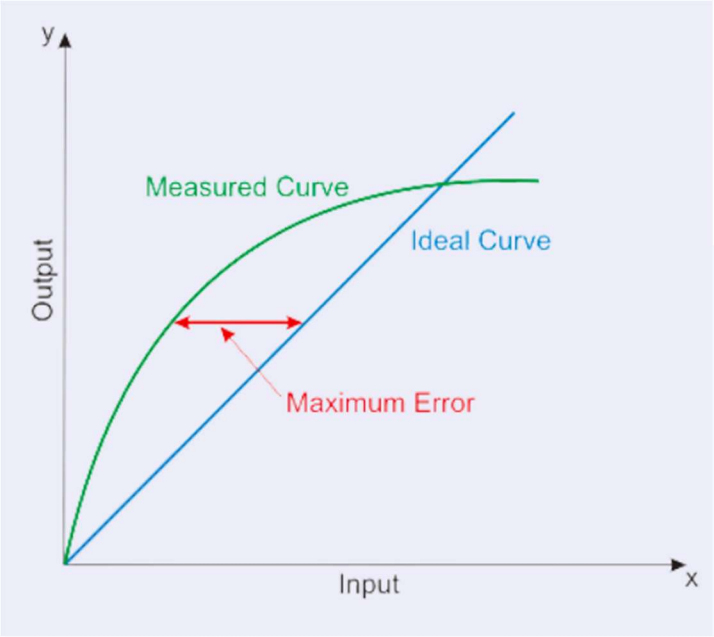
II. MOBILE DEVICE SENSORS *CHARACTERISTICS*

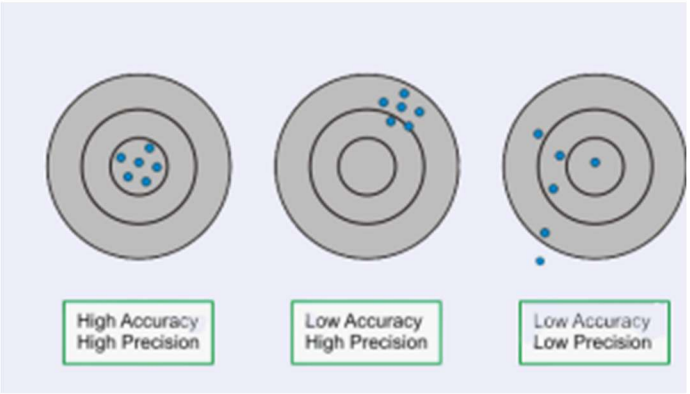
ARE SENSORS ALWAYS CORRECT? *"Numbers don't lie, people do"*

Sensors are characterized depending on the value of many parameters, and their manufacturers. We must understand the underlying sensor to get the right thing we want

RANGE	It is the minimum and maximum value of a physical variable that the sensor can sense or measure

ACCURACY	It is defined as the difference between the measured value and the true value.
SPAN	It is the difference between the maximum and minimum values of input.
UNIT	Unit of measurement
SENSITIVITY	It calculates the sensor's output change concerning a unit change in the input (the measured quantity).
LINEARITY	Linearity is the maximum deviation between the measured values of a sensor from ideal curve
RESOLUTION	The slightest change in the input that the sensor can detect and reliably point out is called resolution





TYPE	Physical sensors relay data from the source they present, while composite sensors generate data from other sensors
REPORTING MODE	Sensors can generate events in different ways called reporting modes (Continues, On-change, One-shot,...)
SAMPLING PERIOD	It is the time difference between two consecutive data collection (frequency)

POWER CONSUMPTION	It is the energy consumption of the sensor when being used or sleep
CALIBRATION	Sensor calibration makes that instrument function as accurately, or error-free, as possible

III. COMMON MOBILE DEVICE SENSORS

1. SENSOR CATEGORY (ANDROID)

Motion Sensors: These sensors measure acceleration forces and rotational forces along three axes. This category includes accelerometers, gravity sensors, gyroscopes, and rotational vector sensors.

Position Sensors: These sensors measure the physical position of a device. This category includes orientation sensors and magnetometers.

Environmental Sensors: These sensors measure various environmental parameters, such as ambient air temperature and pressure, illumination, and humidity. This category includes barometers, photometers, and thermometers.

2. MOTION SENSORS

Motion sensors are useful for monitoring device movement, such as tilt, shake, rotation, or swing. The movement is usually a reflection of direct user input, but it can also be a reflection of the physical environment in which the device is sitting

Sensor	Type	Description	Common Uses
TYPE_ACCELEROMETER	Hardware	Measures the acceleration force in m/s2 that is applied to a device on all three physical axes (x, y, and z), including the force of gravity	Motion detection (shake, tilt, etc.).
TYPE_GRAVITY	Software or Hardware	Measures the force of gravity in m/s2 that is applied to a device on all three physical axes (x, y, z).	
TYPE_GYROSCOPE	Hardware	Measures a device's rate of rotation in rad/s around each of the three physical axes (x, y, and z).	Rotation detection (spin, turn, etc.).
TYPE_LINEAR_ACCELERATION	Software or Hardware	Measures the acceleration force in m/s2 that is applied to a device on all three physical axes	Monitoring acceleration along a single axis

		(x, y, and z), excluding the force of gravity	
TYPE_ROTATION_VECTOR	Software or Hardware	Measures the orientation of a device by providing the three elements of the device's rotation vector.	Motion detection and rotation detection.

3. POSITION SENSORS: useful for determining a device's physical position in the world's frame of reference

Sensor	**Type**	**Description**	**Common Uses**
TYPE_MAGNETIC_FIELD	Hardware	Measures the ambient geomagnetic field for all three physical axes (x, y, z) in μT	Creating a compass.
TYPE_ORIENTATION	Software	Measures degrees of rotation that a device makes around all three physical axes (x, y, z). As of API level 3 you can obtain the inclination matrix and rotation matrix for a device by using the gravity sensor and the geomagnetic field sensor in conjunction with the getRotationMatrix() method	Determining device position.
TYPE_PROXIMITY	Hardware	Measures the proximity of an object in cm relative to the view screen of a device. This sensor is typically used to determine whether a handset is being held up to a person's ear.	Phone position during a call.

4.ENVIRONMENT SENSORS: let you monitor various environmental properties.

Sensor	Type	Description	Common Uses

TYPE_AMBIENT_TEMPERATURE	Hardware	Measures the ambient room temperature in degrees Celsius (°C).	Monitoring air temperatures
TYPE_LIGHT		Measures degrees of rotation that a device makes around all three physical axes (x, y, z). As of API level 3 you can obtain the inclination matrix and rotation matrix for a device by using the gravity sensor and the geomagnetic field sensor in conjunction with the getRotationMatrix() method	Controlling screen brightness
TYPE_PRESSURE		Measures the ambient air pressure in hPa or mbar	Monitoring air pressure changes
TYPE_RELATIVE_HUMIDITY		Measures the relative ambient humidity in percent (%)	Monitoring dewpoint, absolute, and relative humidity.
TYPE_TEMPERATURE		Measures the temperature of the device in degrees Celsius (°C). This sensor implementation varies across devices and this sensor was replaced with the TYPE_AMBIENT_TEMPERATURE sens	Monitoring temperatures.

5.SPECIAL DEVICES

Biometric Sensors	An identification and authentication device, based on a physical attribute such as fingerprints, facial images, iris, and voice recognition
System Sensors	Measures degrees of rotation that a device makes around all three physical axes (x, y, z). As of API level 3 you can obtain the inclination matrix and rotation matrix for a device by using the gravity sensor and the geomagnetic field sensor in conjunction with the getRotationMatrix() method
Camera	The camera contains an image sensor or imager that detects and conveys information used to make an image

GNSS	GNSS stands for Global Navigation Satellite System and is the standard generic term for satellite navigation systems. This term includes e.g. the GPS, GLONASS, Galileo, Beidou, and other regional systems

IV. DEVELOP MOBILE APP WITH DEVICE SENSORS

BEFORE/AFTER USING SENSORS	WHILE USING SENSORS
- Determine which sensors are available on a device. - Determine an individual sensor's capabilities, such as its maximum range, manufacturer, power requirements, and resolution. - Acquire raw sensor data and define the minimum rate at which you acquire sensor data. - Register and unregister sensor event listeners that monitor sensor changes.	- Be sure you choose a delivery rate that is suitable for your application or use case. Sensors can provide data at very high rates. Allowing the system to send extra data that you don't need wastes system resources and uses battery power. - Only gather sensor data in the foreground. Apps running in the background have a restriction that can not access sensor events. - Avoid using deprecated methods or sensor types. Several methods and constants have been deprecated and might return unexpected behaviors. - Try to test as many different kinds of mobile devices as possible

Ch9. Final Review (221)

Câu 1. (L.O.2.2) Loại dữ liệu nào sau đây có thể áp dụng loại bàn phím tuỳ chỉnh trên thiết bị di động? Dữ liệu email.

Câu 2. (L.O.3.3) Rủi ro của việc lưu trữ dữ liệu trên các hệ thống điện toán đám mây (cloud) là gì? **Rủi ro rò rỉ dữ liệu**

Câu 3. (L.O.4.1) Phần mở rộng (extension) của các tập tin React Native viết bằng JavaScript là? **Jsx**

Câu 4. (L.O.2.2) Phát biểu nào dưới đây là không đúng?

A. Việc kiểm tra prototype không nên có lập trình viên tham gia.

B. Chỉ nên chuyển qua bước hiện thực khi đã hoàn thành bước thiết kế giao diện cơ bản.

C. Việc triển khai prototype có thể được thực hiện từ wireframe hoặc mockup.

D. Prototype nên thể hiện toàn bộ hành vi của ứng dụng.

Câu 6. (L.O.1.1) Các kiến trúc MVC, MVP, MVI được phân loại là? **Architectural Patterns.**

Câu 7. (L.O.1.2) Đâu không phải là thế mạnh của các thiết bị di động? **Luôn có kết nối mạng**

Câu 8. (L.O.4.1) Việc hiển thị danh sách (list) bằng phương pháp phân trang (paging) có ưu điểm gì so với phương pháp cuộn vô hạn (infinity scroll)?

A. Hiển thị được nhiều đối tượng hơn. B. Trải nghiệm người dùng trực quan hơn.

C. Hiệu suất ứng dụng tốt hơn. D. Tìm kiếm và lọc dữ liệu nhanh chóng

Câu 9. (L.O.4.3) Tính năng nào sau đây trên iOS không cần yêu cầu xác nhận quyền truy cập từ người dùng trên thiết bị di động?

A. Sử dụng máy ảnh. B. Định vị vị trí chính xác của thiết bị.

C. Tạo một sự kiện mới trong lịch người dùng. **D. Tự động gợi ý OTP từ tin nhắn SMS**

Câu 10. **(L.O.2.3)** Lựa chọn các kỹ thuật thiết kế đã được sử dụng cho giao diện dưới đây:

1. Tối giản dữ liệu (decluttering). **2. Nhấn mạnh thông tin quan trọng.**

3. Chia nhỏ các bước thực hiện. 4. Giảm thiểu thông tin nhập từ người dùng.

Câu 11. (L.O.1.4) Ưu điểm của các ứng dụng dạng Hybrid là? **. Một mã nguồn cho tất cả**

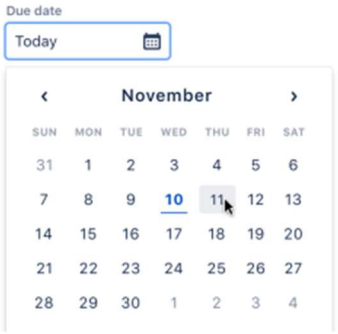
Câu 12. (L.O.1.1) Trong kiến trúc tổng quan của một hệ thống ứng dụng di động, việc hỗ trợ nhiều phiên bản ứng dụng khác nhau (v1, v2,..) nên do thành phần nào đảm trách?

A. Ứng dụng di động. B. Web Service/App Service.

C. Load Balancer/Traffic Manager. **D. Reverse Proxy**

Câu 13. (L.O.2.4) Công cụ nào sau đây không dùng để thiết kế giao diện cho ứng dụng? **Angular**

Câu 14. (L.O.2.3) Tên của thành phần giao diện dưới đây là gì? **Date Picker.**



Câu 15. (L.O.4.1) Đâu là component cơ bản giúp xây dựng layout trên React Native?

A. ViewContainer. B. FlexBox. C. TouchableOpacity. **D. View.**

Câu 16. (L.O.2.2) Đối tượng giao diện CheckBox có thể được phân thành loại nào? **Input Controls**

Câu 17. (L.O.4.3) Dịch vụ nào sau đây cần phải có được sự chấp nhận từ người dùng để có thể được sử dụng?

A. Hiển thị quảng cáo. **B. Truy cập máy ảnh.**

C. Truy cập dữ liệu Internet thông qua 4G/5G. D. Xem góc của thiết bị so với cực Bắc thực (True North)

Câu 18. (L.O.4.4) Việc triển khai ứng dụng di động qua các cửa hàng chính thống đem lại lợi ích gì?

A. Quảng bá rộng rãi sản phẩm. B. Giảm chi phí triển khai ứng dụng.

C. Nhanh chóng đưa sản phẩm tới người dùng. D. Tất cả đều đúng

Câu 19. (L.O.3.1) URI nào dưới đây dùng để lấy thông tin sinh viên có id là 2012345? **/students?id=2012345**

Câu 20. (L.O.2.2) Nguyên tắc chia nhỏ tác vụ thành nhiều bước nhằm?

A. Đảm bảo tính chính xác của các tác vụ liên quan tới dữ liệu quan trọng

B. Giúp giảm thời gian hiện thực tính năng.

C. Giúp người dùng nhanh chóng hoàn thành tác vụ.

D. Tất cả đều đúng.

Câu 21. (L.O.4.2) Vai trò của action trong Redux là? **Tạo ra các sự kiện tương ứng với sự tương tác từ người dùng.**

Câu 22. (L.O.3.2) Định dạng dữ liệu nào sau đây thường được trao đổi qua REST API? **JSON**

Câu 24. (L.O.4.4) Việc khai thác dữ liệu các công cụ phân tích như Firebase Analytics hay Sentry được thực hiện ở giaI đoạn nào? **Triển khai ứng dụng**.

Câu 25. (L.O.1.1) Đối với kiến trúc MVI, hành vi nào sau đây có thể được coi là một Intent?

A. Đăng ký lắng nghe thông tin từ cảm biến. B. Đóng ứng dụng.

C. Kích hoạt một khung nhập dữ liệu. **D. Yêu cầu tải lại dữ liệu.**

Câu 26. (L.O.3.1) Phương thức PUT được coi là idempotent bởi vì?

A. Phương thức PUT chỉ lấy dữ liệu về mà không can thiệp vào trạng thái hệ thống

B. REST luôn đảm bảo chỉ có duy nhất một lần gọi phương thức PUT được thực hiện

C. Mỗi lần gọi lại phương thức PUT đều có hành vi giống nhau với những payload khác nhau

D. Mỗi lần gọi lại phương thức PUT đều có hành vi giống nhau với những payload giống nhau

Câu 27. (L.O.1.4) Tập tin app.config.js trên nền tảng Expo dùng để làm gì?:

A. Chứa các thiết lập khi đóng gói các tập tin JavaScript sang thành JSBundle.

B. Chứa các thiết lập khi khởi chạy hoặc biên dịch ứng dụng.

C. Chứa các thiết lập của ứng dụng khi hoạt động như ngôn ngữ, màu sắc.

D. Tất cả các đáp án trên đều sai.

Câu 28. (L.O.2.2) Phát biểu nào dưới đây là không đúng

A. Việc triển khai prototype có thể được thực hiện từ wireframe hoặc mockup.

B. Chỉ nên chuyển qua bước hiện thực khi đã hoàn thành bước thiết kế giao diện cơ bản.

C. Prototype nên thể hiện toàn bộ hành vi của ứng dụng.

D. Việc kiểm tra prototype không nên có lập trình viên tham gia.

Câu 29. (L.O.4.3) Chế độ thu nhận nào sau đây phù hợp khi đọc dữ liệu từ cảm biến gia tốc? **Continous**

Câu 30. (L.O.1.4) Việc lựa chọn nền tảng Expo để phát triển ứng dụng React Native đem lại lợi ích gì so với React Native thuần.? . **Không cần thiết phải cài đặt Android/iOS SDK**

Câu 31. (L.O.3.2) Ưu điểm của JSON so với XML là?

A. Kích thước nhỏ hơn. **B. Dễ đọc hiểu hơn.** **C. Được sử dụng rộng rãi hơn**

Câu 32. (L.O.2.3) Phát biểu nào sau đây là sai?

A. Việc xây dựng Design System giúp giảm bớt thời gian cập nhật khi yêu cầu bị thay đổi.

B. Các plugin trên Figma giúp đẩy nhanh quá trình thiết kế giao diện ứng dụng.

C. Cần phải xây dựng prototype để đảm bảo ứng dụng hoạt động đúng theo nhu cầu.

D. Phải thiết kế hoàn chỉnh tất cả giao diện của ứng dụng trước khi bước vào giai đoạn hiện thực

Câu 34. **(L.O.4.1)** Tính năng nào dưới đây nên được lựa chọn khi xây dựng phiên bản MVP cho một ứng dụng mạng xã hội? **Danh sách bạn bè**

Câu 36. **(L.O.2.3)** Việc thực hiện Wireframe được thực hiện ở bước nào trong quy trình thiết kế giao diện ứng dụng?

A. Create. B. Research. C. Test. D. Empathize

Câu 38. (L.O.4.3) Phát biểu nào sau đây là đúng? **Việc sử dụng liên tục cảm biến có thể tiêu tốn nhiều năng lượng**

Câu 39. **(L.O.2.3)** Việc thiết kế giao diện ở dạng Wireframe đem lại lợi ích gì?

A. Kiểm tra khả năng tương tác từ người dùng. B. Tăng mức độ chi tiết.

C. Kiểm tra tính năng của ứng dụng. D. Tất cả các ý trên

Câu 40. **(L.O.4.1)** Việc sử dụng callEffect() hook trên React không thay thế được hàm nào trong vòng đời của một React class?

A. componentDidUpdate(). B. componentWillUnmount(). **C. componentWillMount().** D. componentDidMount()

Câu 41. **(L.O.1.3)** Đặc điểm nào sau đây cần chú ý đặc biệt khi hỗ trợ lâu dài các ứng dụng trên thiết bị di động?

A. Dung lượng pin của thiết bị theo thời gian. B. Chi phí cho việc duy trì ứng dụng trên cửa hàng.

C. Sự thay đổi của phần cứng trên thiết bị. **D. Các phiên bản cập nhật của hệ điều hành.**

Câu 42. (L.O.1.1) Phát biểu nào sau đây là đúng khi nói về các lớp (layer) trong SoC?

A. Các lớp được liên kết lỏng lẻo.

B. Các lớp được phân chia theo chiều ngang.

C. Khi một lớp thay đổi sẽ không ảnh hưởng tới các lớp khác.

D. Tất cả đều đúng

Câu 43. (L.O.1.4) Nền tảng React Native có thể được phân loại là? **. Cross-platform App**

Câu 44. (L.O.2.2) Đâu là sự khác biệt giữa quá trình tương tác từ người dùng tới ứng dụng trên máy tính để bàn và tới ứng dụng trên thiết bị di động? **Phản hồi vật lý.**

Câu 45. (L.O.2.1) Đâu là sự khác biệt lớn giữa việc thiết kế giao diện cho ứng dụng trên máy tính và ứng dụng trên thiết bị di động? **Tỉ lệ màn hình**.

Câu 46. (L.O.4.2) Công việc nào nên được thực hiện khi một thành phần (component) trên giao diện sắp bị huỷ (destroy)?

A. Xoá dữ liệu người dùng. B. Tải lại toàn bộ dữ liệu.

C. Tính toán lại layout của component đó. **D. Huỷ đăng ký các callback/listener đang sử dụng.**

Câu 47. (L.O.4.1) Hàm nào không nằm trong vòng đời của một React class?

A. componentWillRender(). B. render(). C. componentWillMount(). D. componentDidUpdate()

Câu 48. (L.O.2.3) Ưu điểm của việc hiển thị biểu tượng (icon) bằng các bộ font vector so với bằng tập tin hình ảnh (jpeg/png) là? **Tốc độ hiển thị nhanh chóng**

Câu 49. (L.O.1.3) Nhân tố nào **không dùng** để đánh giá sự thành công của một mobile app **Quy trình phát triển**

Câu 50. (L.O.3.2) Yêu cầu nào dưới đây là bắt buộc khi xây dựng một Web Service cho nền tảng iOS?

A. Dùng giao thức HTTPS. B. Không được lưu trữ thông tin cá nhân của người dùng.

C. Phải sử dụng công cụ từ Apple. D. Chỉ được giao tiếp với các Web Service trên Internet

Câu 51. (L.O.1.1) Đâu là 3 lớp chính trong kiến trúc ứng dụng cho thiết bị di động?

A. Presentation, Business, Data. B. Data View, Model, Controller.

C. Presentation, Data, Model. D. Presentation, Controller, Data.

Câu 52. (L.O.4.4) Nhược điểm của việc cập nhật OTA các ứng dụng React Native là?

A. Tốc độ cập nhật chậm. B. Phải chờ xét duyệt từ cửa hàng.

C. Không cập nhật được các thành phần liên quan tới tầng native. D. Phải được sự cho phép từ người dùng.

Câu 53. (L.O.4.4) Công cụ quản lý việc xuất bản các ứng dụng Android là gì? . **Google Play Console**

Câu 54. (L.O.1.2) Lý do nào dưới đây khiến cho việc phát triển ứng dụng trên nền tảng thiết bị di động thường tốn nhiều thời gian hơn so với việc phát triển ứng dụng trên nền tảng máy tính cá nhân? **Số lượng thiết bị cần kiểm thử**

Câu 55. (L.O.4.4) Chi phí cho tài khoản Nhà phát triển để phát hành ứng dụng Android là? . **25 US$**

Câu 56. (L.O.2.2) Sự kiện nào sau đây cần chú ý riêng khi thiết kế UI/UX cho ứng dụng di động? **Kết nối mạng không ổn định.**

Câu 57. (L.O.3.3) Chọn các phương pháp dưới đây giúp bảo mật dữ liệu khi trao đổi và lưu trữ bằng Web Service:

1. Sử dụng giao thức SSL cho kết nối.

2. Áp dụng quy chuẩn RESTful khi thiết kế API.

3. Mã hoá thông tin cá nhân của người dùng trên cơ sở dữ liệu.

4. Thường xuyên cập nhật các bản vá bảo mật trên công cụ phát triển ứng dụng.

Câu 58. (L.O.3.1) Phương thức HTTP nào không có trong RESTful? **SEARCH**

Câu 59. (L.O.2.2) Việc áp dụng màn hình Onboarding giúp giải quyết vấn đề gì**? Giúp người dùng nắm được các tính năng cốt lõi của ứng dụng**

Câu 60. (L.O.3.4) Công cụ giúp lập trình viên tự tạo tài liệu đặc tả API có tên là? Swagger

Ch9. Final Review (222)

Câu 1. (L.O.1.1) Việc lựa chọn nền tảng Expo để phát triển ứng dụng React Native có hạn chế gì so với việc sử dụng React Native thuần?

Kích thước ứng dụng lớn hơn so với ứng dụng React Native tương đương.

Câu 2. (L.O.1.1) 2. Sự khác biệt lớn nhất của ứng dụng dạng Hybrid và PWA (Progressive Web App) là?

Cách đóng gói và triển khai ứng dụng

Câu 3. (L.O.1.2) Ứng dụng nào sau đây nên được phát triển song song trên cả ứng dụng di động và thiết bị đeo thông minh?

Ứng dụng chạy bộ.

Câu 4. (L.O.1.3) Lợi ích nào dưới đây **không phải** của ứng dụng trên thiết bị di động? Tốc độ triển khai sản phẩm nhanh chóng.

Câu 5. (L.O.1.4) Việc lựa chọn nền tảng Expo để phát triển ứng dụng React Native có hạn chế gì so với việc sử dụng React Native thuần? Kích thước ứng dụng lớn hơn so với ứng dụng React Native tương đương.

Câu 6. (L.O.1.4) Sự khác biệt lớn nhất của ứng dụng dạng Hybrid và PWA (Progressive Web App) là

Cách đóng gói và triển khai ứng dụng

Câu 7. (L.O.2.1) Giao diện cho ứng dụng trên thiết bị di động thường được tối ưu hoá cho?

Màn hình nhỏ và cảm ứng.

Câu 8. (L.O.2.2) Trên thiết bị di động, phím điều hướng thường được thay thế bằng? Màn hình cảm ứng.

Câu 9. (L.O.2.2) Điều gì làm cho giao diện ứng dụng cho máy tính khác biệt so với giao diện trên thiết bị di động?

A. Kích thước các nút và biểu tượng lớn hơn.
B. Có thể truy cập vào bàn phím số.
C. Thông tin hiển thị nhiều hơn trên một màn hình.
D. Tất cả đều đúng.

Câu 10. (L.O.2.2) Giao diện ứng dụng cho máy tính thường cung cấp các chức năng nào mà giao diện cho thiết bị di động **không có**? Phím tắt.

Câu 11. (L.O.2.2) Tại sao các hoạt động trên ứng dụng di động thường có thời gian ngắn? Tính di động của thiết bị.

Câu 12. (L.O.2.3) Yếu tố nào sau đây cần được cân nhắc khi thiết kế giao diện người dùng (UI) cho ứng dụng di động? Kích thước màn hình và độ phân giải hiển thị.

Câu 13. (L.O.2.3) Mục đích của việc thực hiện bước nghiên cứu người dùng (User Research) trong quy trình thiết kế UI/UX là gì?

A. Hiểu rõ sở thích và nhu cầu người dùng. **B.** Giảm thiểu kiểm thử ở người dùng.

C. Làm giảm độ phức tạp của ứng dụng. **D.** Tăng tốc quá trình phát triển.

Câu 14. (L.O.2.4) Công cụ Balsamiq thường dùng để thiết kế giao diện ở mức độ nào? Wireframe.

Câu 15. (L.O.2.3) Khía cạnh nào của thiết kế UI/UX trên ứng dụng di động tập trung vào việc tạo kết nối cảm xúc với người dùng?

A. Xây dựng hệ thống thiết kế chung (Design System) cho ứng dụng.

B. Định nghĩa các chỉ số (Metrics) đánh giá hiệu suất ứng dụng.

C. Nghiên cứu hành trình trải nghiệm người dùng (Experience Map) trên ứng dụng.

D. Đảm bảo đáp ứng các quy chuẩn (Guideline) cho người khuyết tật.

Câu 16. (L.O.3.2) Việc định nghĩa kiểu dữ liệu trong HTTP Request được khai báo ở đâu?

A. Request Body. **B.** Request Header. **C.** Request Message. **D.** Request Target.

Câu 17. (L.O.3.1) Mục đích của việc sử dụng các mã trạng thái (HTTP Status Code) trong Web Service? Để xử lý ngoại lệ và lỗi khi thực thi yêu cầu.

Câu 18. (L.O.3.1) Phương thức HTTP nào thường được sử dụng để tạo tài nguyên mới tại URI được chỉ định? POST

Câu 19. (L.O.3.3) Lợi ích **chính** của việc thiết lập **lưu trữ dữ liệu trên máy chủ/trên đám mây** so với **trên thiết bị di động** là gì?

A. Khả năng truy cập dữ liệu được mở rộng. **B.** Tốc độ truy cập dữ liệu nhanh hơn.

C. Tăng cường tính bảo mật của dữ liệu. **D.** Giảm thiểu chi phí lưu trữ dữ liệu lâu dài.

Câu 20. (L.O.3.4) Module nào chịu trách nhiệm là Root Module trong NestJS?

A. AppModule. **B.** MainModule. **C.** RootModule. **D.** NestModule.

Câu 21. (L.O.3.2) Định dạng dữ liệu nào sau đây dựa trên ngôn ngữ đánh dấu (Markup Language) và sử dụng các thẻ (Tag) để xác định các thành phần và thuộc tính? XML (eXtensible Markup Language).

Câu 22. (L.O.4.1) Đâu là component cơ bản giúp định hình giao diện trên React Native? View.

Câu 23. (L.O.4.2) Redux giải quyết vấn đề gì trong quá trình phát triển ứng dụng React Native? Xử lý luồng chia sẻ dữ liệu trên hệ thống phân cấp các component.

Câu 24. (L.O.4.4) Để triển khai cập nhật qua OTA cho các ứng dụng React Native dựa trên Expo, chúng ta sử dụng dịch vụ nào?

A. Expo Application Services. **B.** Firebase OTA. **C.** AppStore Connect. **D.** Expo CLI Update.

Câu 25. (L.O.4.3) Quyền ứng dụng (App Permission) trong ngữ cảnh ứng dụng di động là gì?

Thường xuyên xem xét và cập nhật các quyền của ứng dụng.

Câu 26. (L.O.1.1) Việc lựa chọn kiến trúc triển khai ứng dụng trên thiết bị di động ảnh hưởng đến khả năng mở rộng và sự phát triển trong tương lai của sản phẩm như thế nào?

Cung cấp nền tảng để thêm các tính năng mới theo thời gian.

Câu 27. (L.O.4.4) Quy trình xem xét ứng dụng (App Review) trên các cửa hàng ứng dụng là?

Nhân viên kiểm duyệt sẽ kiểm tra và phê duyệt ứng dụng để phân phối trên cửa hàng ứng dụng.

Câu 28. (L.O.4.3) Hành động nào có thể giúp bảo vệ quyền riêng tư của người dùng khi sử dụng ứng dụng dành cho thiết bị di động? Thường xuyên xem xét và cập nhật các quyền của ứng dụng.

Câu 29. (L.O.4.1) Việc định nghĩa các thuộc tính style cho các component trong React Native có thể được thực hiện như thế nào? Inline style bằng style prop.

Câu 30. (L.O.4.4) Công cụ quản lý việc xuất bản các ứng dụng di động do Apple cung cấp là gì? App Store Connect.

Câu 31. (L.O.4.1) Trên React Native, chúng ta có thể lồng các component View với nhau hay không?

A. Có. Không có giới hạn về số lần lồng nhau.

B. Không. Các component View chỉ có thể có nhiều nhất một cấp độ lồng nhau.

C. Có. Nhưng điều này không được khuyến khích vì vấn đề hiệu năng ứng dụng.

D. Không. Các component View chỉ có thể được lồng trong các thành phần khác.

Câu 33. (L.O.3.1) Mục đích chính khi triển khai các Web Service dựa trên REST là gì?

A. Tương tác dựa trên tài nguyên. **B.** Tin nhắn theo thời gian thực.

C. Tái sử dụng các thành phần. **D.** Quản lý cơ sở dữ liệu.

Câu 34. (L.O.4.3) Cảm biến nào thường được sử dụng để đếm bước và theo dõi hoạt động trong thiết bị di động?

Accelerometer

Câu 35. (L.O.4.3) Mục đích của cảm biến tiệm cận trong thiết bị di động?

Phát hiện các đối tượng ở gần và điều chỉnh độ sáng màn hình.

Câu 36. (L.O.2.2) React Native component nào được sử dụng để hiển thị danh sách đối tượng có thể cuộn?

A. ScrollView **B.** ListView. **C.** FlatList. **D.** Information Components.

Câu 37. (L.O.2.2) Việc loại bỏ các trường thông tin không cần thiết khi đăng ký tài khoản thuộc hướng dẫn thiết kế giao diện nào dưới đây trên ứng dụng di động? Giảm thiểu yêu cầu dữ liệu đầu vào từ người dùng.

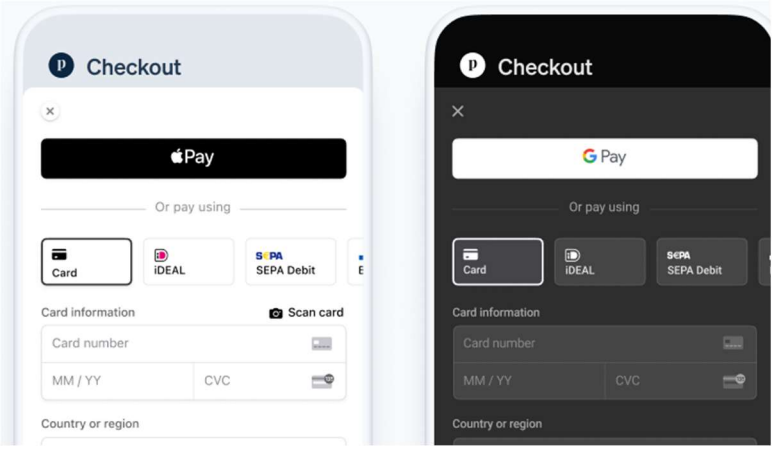
Câu 38. (L.O.1.1) Layer nào không có trong kiến trúc chung của ứng dụng di động? Middleware Layer.

Câu 39. (L.O.4.4) Tên công cụ thu thập lỗi thường được sử dụng trên các ứng dụng React Native? Sentry

Câu 40. (L.O.4.2) Thành phần có thể thay thế các hàm Life Cycle trên React Class là?

A. Presentation Layer. **B.** Business Layer. **C.** Data Layer. **D.** Middleware Layer.

Câu 42. (L.O.2.3) Lựa chọn các kỹ thuật thiết kế đã được sử dụng cho giao diện dưới đây:



1. *Tối giản dữ liệu (decluttering).*
2. *Chia nhỏ các bước thực hiện.*

2. Nhấn mạnh thông tin quan trọng.

4. Giảm thiểu thông tin nhập từ người dùng.

Câu 43. (L.O.1.3) Nhân tố nào dưới đây không phải đặc trưng của thiết bị di động?

A. Thời lượng pin có hạn. **B.** Triển khai qua cửa hàng.

C. Cập nhật chậm. **D.** Kich thước màn hình nhỏ.

Câu 44. (L.O.1.4) Ưu điểm của các ứng dụng dạng Web View so với Hybrid là? Không cần cài đặt trên thiết bị.

Câu 45. (L.O.1.4) Nền tảng React Native có thể được phân loại là? Cross-platform App.

Câu 46. (L.O.2.2) Điều gì làm cho giao diện ứng dụng trên thiết bị di động dễ sử dụng hơn đối với người dùng?

Giao diện tương tác đơn giản và trực quan.

Câu 47. (L.O.2.3) Tên của thành phần giao diện dưới đây là gì? 

A. Toggle. **B.** Checker. **C.** Swiper. **D.** Check Box.

Câu 49. (L.O.2.2) Mục đích của việc tạo prototype trong quy trình thiết kế UI/UX là gì??

Để kiểm tra khả năng sử dụng và chức năng của thiết kế với người dùng thực.

Câu 50. (L.O.1.1) Thành phần Security nằm ở lớp nào trong kiến trúc ứng dụng di động?

A. Business Layer. B. Cross-Cutting Layer.

C. Data Layer. **D.** Presentation Layer.

Câu 51. (L.O.3.2) Định dạng dữ liệu nào thường được sử dụng để trao đổi dữ liệu giữa các dịch vụ web? JSON

Câu 53. (L.O.4.1) Phần mở rộng (extension) của các tập tin React Native viết bằng TypeScript là? .tsx

Câu 54. (L.O.3.3) Đâu là một nền tảng đám mây để lưu trữ dữ liệu? Amazon Web Service

Câu 55. (L.O.2.3) Việc thực hiện Prototype được thực hiện ở bước nào trong quy trình thiết kế giao diện ứng dụng?

A. Research. B. Create. **C.** Empathize. **D.** Test.

Câu 56. (L.O.4.1) React Native là? Framework

Câu 57. (L.O.4.1) Việc sử dụng "props" trong React Native có tác dụng gì?

A. Truyền dữ liệu xuống component con. **B.** Định nghĩa trạng thái của component.

C. Xử lý tương tác của người dùng. **D.** Định nghĩa giao diện của component.

Câu 58. (L.O.2.2) Công việc nào dưới đây nên chia nhỏ thành nhiều bước khi triển khai trên ứng dụng di động?

A. Thanh toán đơn hàng. **B.** Đăng ký tài khoản.

C. Đăng xuất tài khoản. **D.** Tìm kiếm danh sách sản phẩm.

Câu 59. (L.O.1.2) Đâu không phải là đặc trưng của các thiết bị di động? Truy cập mạng không giới hạn.

Câu 60. (L.O.3.1) Loại kiến trúc nào thường được kết hợp với REST? Client-Server.

QUIZ

Week 2.

Câu 1. Which reason does not promote the mobile application industry?

A. People spend more time on their smartphones B. Many people own a smartphone today

C. Smartphone price is decreased D. New hardware technology is introduced each year

E. Old smartphones are recycled more efficiently

Câu 2. Which answer describes the main characteristics of mobile devices?

A. Stable Communication, Store Distribution, Screen Size, Battery Life, Small Updates

B. Battery Life, Stable Changes, Store Distribution, Broad Communication, Small Screen Size

C. Battery Life, Continuous Changes, Store Distribution, Unstable Communication, Various Screen Size

D. Store Distribution, Small Screen Size, Battery Life, Continuous Changes, Unstable Communication

Câu 3. How many percentages (__%) do people spend on Entertainment on Mobile phones? 17%

Câu 4. Match between development language and mobile app platform

JavaScript → React Native, C# → Xamarin, Dart → Flutter, Kotlin → Android, Swift → iOS, CSS, HTML5 → Ionic

Câu 5. Which process suits the Minimum Viable Product (MVP) definition while building a new house?

A. Start from floors, pillars, walls then roofs

B. Buy a whole constructed house

C. Build a small house first will required rooms, then expand the house by rebuilding all rooms

D. Start from bedroom, kitchen to bathroom, living room and then garage

Week 3.

Câu 1. Which feature is not an MVP for the Walking Buddy Finder App?

A. Track Planning **B. User Support** C. Buddy Lookup D. Login E. Account Registration

Câu 2. Is this order of complexity in UI/UX design correct? => **False**.

Wireframe < Mockup < Prototype

Câu 3. Please match the corresponding SWOT statement of the MyBK Mobile App

Weaknesses → Business expansion across Ho Chi Minh city,

Strengths → Integrate with mobile device features such as Camera, Calendar,

Threats → Data Access Permission. Opportunities → Number of HCMUT students

Câu 4. Why the Empathize step is important?

A. Because it helps developers collect sufficient metrics to improve user experience

B. Because it helps designers specify user requirements and product outcomes

C. Because it helps designers undestand users' perspective

D. Because it helps our product become profitable

Câu 5. Which factor does not determine a good mobile app?

A. User Experience **B. Customer Receptiveness** C .Business Goals D. Privacy E. Security

WEEK 4

Câu 1. Which is the benefit of rasterized images (JPG/PNG) over vector images (SVG)?

A. Render Quality B. File Size
C. Modification **D. Popularity**

Câu 2. This type of keyboard is suitable for?

A. OTP Password B. Address
C. Email D. Currency

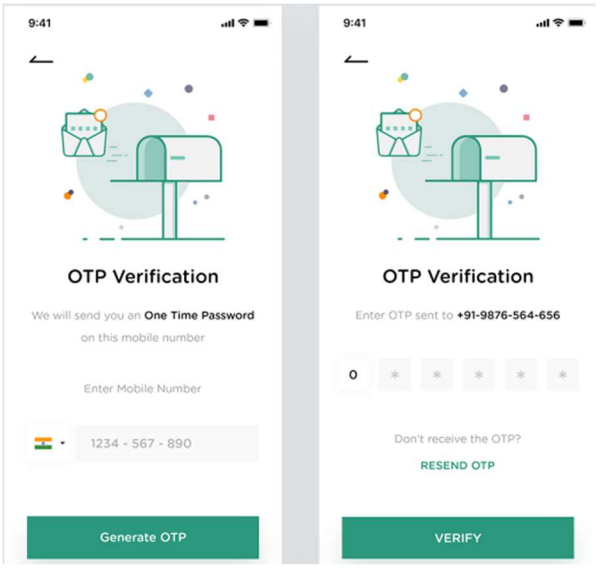


Câu 3. Please match each input with the corresponding category

Date Picker → Input Control, Bottom Bar → Navigation Component,
Modal Dialog → Container, Step Indicator → Information Component

4. Which guideline is applied here?

A. Minimize User Input
B. Make Onboarding Efficient
C. Declutter
D. Breakdown Tasks



5. Name of this component?



A. Button **B. Badge**
C. Tooltips D. Label E. Tags

WEEK 5

Câu 1. Which component is not a part of a modern mobile app system?

A. Push Notification Service B. In-Memory Cache
C. Load Balancer D. App Analytics **E. None of the above**

Câu 2. Which statement is incorrect about React Redux?

A. React components subscribe changes from Redux Store to update the U
B. You can not directly modify the Redux Store
C. Redux pattern is only applied in React applications
D. Actions will be processed by Reducers to modify the app state

Câu 3. The main benefit of the MVVM pattern over the MVP pattern?

A. Improve testing performance B. Improve app performance
C. Reduce app complexity D. Reduce the number of app layers

Câu 5. MVC, MVP, and MVI are?

A. Architectural Styles B. System Architecture C. Design Patterns **D. Architectural Patterns**

WEEK 6

Câu 1. Match each REST method with a corresponding data operation

PUT → UPDATE, GET → READ, POST → CREATE, DELETE → DELETE

Câu 3. What does the following URI "/orders?limit=25&offset=50" mean ?

A. Select the first 50 items B. Retrieve the first 25 orders
C. Choose orderly 50 orders from the 25th order **D. Pick orderly 25 orders from the 50th order**

Câu 4. Assume we are designing a Smart Home app to manage multiple houses. To select a device inside a house, which REST URI is correct?

A. /houses/devices/{device-id} B. /houses/{house-id}/device
C. /houses/{house-id}/devices/{device-id} D. /houses/{house-id}/device/{device-id}
E. /house/{house-id}/device/{device-id}

Câu 5. When do we prefer SOAP over JSON?

A. We need to reduce packet size **B. We need an embedded security schema**
C. We need to support various types of data D. We need to cache popular data

WEEK 7

Câu 1/ We can test our app using which app?

A. Expo Client B. Expo Mobile C. Expo Mirror D. Expo Go E. Expo for Student

Câu 2/Which company is behind the success of React Native?

A. Alibaba **B. Facebook** C. Google D. Amazon

Câu 3/ We can build our Expo project into APK without Android SDK installed on our machine, it this true?

True, **Use EAS build**

Câu 4/ Which command that we use to create a new Expo project?

A. npx create-expo-app my-app B. npx create-react-app my-app
C. npm create-react-native-app my-app D. npx create-react-native-app my-app

Câu 5. Which reason can explain why we choose React Native?

A. The lowest investment costs for moving from Web platforms B. Its performance is super fast
C. A large community and available libraries D. A long history and official development toolkits

WEEK 8

Câu 1. When can we move a local state value to the global state store?

A. It is fetched from external sources such as a REST API **B. It is shared between different screens and components**
C. It is stored inside the Redux Store D. It must be initialized after the app starts
E. The value is changed by an interactive event such as touching a button

Câu 2. When a component/activity/scene will be removed from the current screen, we should perform which action?

A. Unbind used callbacks or listeners B. Update component layouts (height/width,...)
C. Process retrieved data D. Perform Rest API requests

Câu 3. The main purpose of Mobile App life cycle callbacks is?

A. Reduce app development efforts **B. Reduce app crashing and power consumption**
C. Manage app state easily D. Improve User Interface

Câu 4. Metrics that we collect from the device's sensors can be categorized as?

A. Local State B. Global State **C. Environment State** D. App State E. Sensor State

Câu 5. Which is the role of the Redux reducer?

A. Update the Redux global state based on dispatched actions
B. Hold the global state which is shared between components
C. Pick required data to render the UI
D. Emit corresponding events based on received user interactions

WEEK 9

Câu 1. To authenticate a user after opening a mobile app (assume that this user has logged in before), which information should be stored?

A. Public Key B. Username and password **C. Access Token** D. Private Key E. Cookies

Câu 2. In a mobile app, when should we use animations?

A. Indicate which fields are incorrectly typed B. Render retrieved data from a REST API

C. Perform background tasks such as data encryption **D. Display an information modal**

Câu 3. Which animation file type is mostly used before the vector-based animation? **GIF**

Câu 4. Where should we store the required data to automatically authenticate users after reopening mobile apps?

A. App Local Storage B. Backup Folders C. Server Databases D. External Storage E. System Shared Storage

Câu 5. Assume that a user had performed an action and an error occurred. Which HTTP status code will redirect this user to the Login page? A. 500 **B. 401** C. 403 D. 404 E. 302

Week 10

Câu 1. What is the main difference between the TYPE_AMBIENT_TEMPERATURE sensor and the TYPE_TEMPERATURE sensor?

A. The TYPE_TEMPERATURE is available on all devices

B. TYPE_AMBIENT_TEMPERATURE is more accurate

C. TYPE_AMBIENT_TEMPERATURE measures the surrounding temperature

D. The TYPE_TEMPERATURE type is deprecated

Câu 2. If our app is using a deprecated sensor API, what should we do?

A. Buy a new phone B. Just leave it as is. Sensor APIs are stable and not subjected to change.

C. Check whether the old API is working or not. If yes, leave it as is.

D. Use another library/framework. **E. Always update to the latest API.**

Câu 4, Which sensor must be used (on Android)?

A. TYPE_ACCELEROMETER **B. TYPE_STEP_DETECTOR**

C. TYPE_GRAVITY D. TYPE_STEP_COUNTER E. TYPE_PRESSURE

Câu 5. Which sensor requires the calibration step before being able to use?

A. TYPE_ACCELEROMETER **B. TYPE_MAGNETIC_FIELD**

Week 11.

Câu 1. What is the suitable reporting mode of the step detecting sensor?

A. On Change B. On-demand C. One Shot D. Continuous

Câu 2. Which statement is correct?

A. Embedded sensors are always turn off on smartphones to save energy

B. Recored sensor values are promised to be the same on different devices

C. Modern smartphones only use a couple of sensors

D. Developer can utilize device sensors to improve user experience.

Câu 3. *"A light sensors can read a value from 0 to 50.000 lx"*

Which characteristics are described in this statement?

A. Accuracy, Unit B. Range, Span C. None is correct **D. Range, Unit** E. Span, Unit

Câu 4. Which interval is suitable for collecting data from a heart rate sensor?

A. 100ms **B. 1min** C. 1s D. 1ms E. 10ms

Câu 5. Which is the main difference between composite (synthetic) and physical sensors?

A. Composite sensors can be installed and removed from the device

B. Synthetic sensors values are inferred from multiple underlying physical sensors

C. Composite sensors always require much energy

D. Synthetic sensors provide more accurate values than physical sensors

E. Working with composite sensors requires additional libraries and techniques